



**RIGA NORDIC
UNIVERSITY**

Studiju programma

42484 Informācijas sistēmas (bak.)

Dabaszinātņu un datortehnoloģiju katedra

**RĪGAS ZIEMEĻVALSTU UNIVERSITĀTE
(RNU)**

**Starpplatformu kases un naudas pārvedumu platformas
„Ethnocount” projektēšana un izstrāde nelielām valūtas maiņas
tīkliem, izmantojot Flutter un Supabase**

BAKALaura DARBS

Students:

Farruhs Muzrabovs

Darba vadītājs:

Dr.sc.ing., prof. Jeļena Čaiko

RĪGA 2026

Study Programme

42484 Information Systems (BSc)

Department of Natural Sciences and Computer Engineering

**RIGA NORDIC UNIVERSITY
(RNU)**

**Design and Implementation of a Cross-Platform Treasury and
Money-Transfer Platform (“Ethnocount”) for Small
Currency-Exchange Networks Using Flutter and Supabase**

BACHELOR’S THESIS

Student: Farruh Muzrabov

Supervisor: Dr.sc.ing., Prof. Jelena Chayko

Anotācija

Daudzās bijušās Padomju Savienības valstīs darbojas nelieli biroji, kas maina vienu valūtu pret citu un nosūta naudu uz citām valstīm. Piemēram, ģimene Uzbekistānā var ienākt šādā birojā, lai saņemtu naudu, ko atsūtījis radnieks, kurš strādā Krievijā, un birojs reizi nedēļā norēķinās ar partneru biroju Maskavā. Šādi biroji bieži ir ģimenes uzņēmumi ar filiālēm vairākās pilsētās un valstīs. Tie ikdienā apgroza ievērojamas naudas summas, taču parasti glabā ierakstus papīra žurnālā vai Excel izklājlapā. Tāpēc kļūdas notiek bieži, un viena izlaista rinda var radīt zaudējumu vairāku tūkstošu ASV dolāru apmērā, kura izsekošanai vēlāk nepieciešamas vairākas nedēļas.

Kļūdas notiek tāpēc, ka birojā patiesībā vienlaikus ir jāuztur trīs dažādu veidu uzskaites. Pirmā ir nauda paša biroja kasē un uz paša biroja kartēm. Otrā ir tekošais parāds ar katru partneru biroju ārzemēs, un šis parāds ir jāseko vairākās valūtās vienlaikus, jo vienam un tam pašam partnerim vienlaikus var būt parāds gan Krievijas rubļos, gan Turcijas lirās. Trešā ir pastāvīgo klientu nauda, kas bieži uzkrāj birojā nelielas summas dažādās valūtās. Standarta grāmatvedības programmas visus trīs šos veidus apvieno vienā skaitlī vienā valūtā, un tas tieši slēpj to, kas birojā patiešām jāredz.

Šajā bakalaura darbā ir aprakstīta programma *Ethnocount*, ko autors izstrādājis šīs reālās problēmas risināšanai. Programma strādā uz telefoniem (Android, iPhone) un datoriem (Windows, macOS, Linux) no vienota koda, tāpēc filiāle var lietot to pašu rīku neatkarīgi no tā, vai operators sēž pie galda vai pārvietojas ar telefonu rokā. Programma trīs uzskaites veidus saglabā atsevišķi — tieši tā, kā tie patiesībā pastāv; tā pareizi summē naudu pat tad, ja vienam partnerim ir parādi trīs dažādās valūtās; un tā nodrošina, ka pārvedumu nevar nejauši saglabāt divreiz, pat ja operators sliktā interneta savienojuma dēļ nospiež pogu divas reizes. Pastāv arī noteikumi par to, kurš ko drīkst darīt: tīkla dibinātājs redz visu, direktors redz visas filiāles, bet parastais darbinieks redz tikai savu filiāli un pirms pagātnes ierakstu maiņas ir jāprasa direktora atļauja.

Autors uzbūvēja sistēmu, rūpīgi pārbaudīja to ar daudziem automātiskiem testiem, izmērīja, cik ātri tā darbojas uz parasta Windows datora un uz vidējas klases Android telefona, un izmēģināja to divas nedēļas kopā ar reālu maiņas biroju. Izmēģinājuma laikā birojs uzturēja gan veco Excel izklājlapu, gan jauno programmu, un katras nedēļas beigās tās tika

salīdzinātas. Jaunā programma neradīja nevienu savu kļūdu. Gluži pretēji, tā atrada trīs vecas kļūdas izklājlappā, kuras operators iepriekš nebija pamanījis.

Darbam joprojām ir ierobežojumi, kas tiek godīgi izklāstīti. Programma vēl nepatstāvīgi neielādē valūtas kursus no interneta — operatoram tie jāievada manuāli. Tā neļauj operatoram saglabāt pārvedumu, kad nav interneta savienojuma — šādā gadījumā darbība tiek atteikta, jo ar naudu drošāk ir atteikt nekā riskēt ar nepareizu kārtību vēlāk. Un tā tika pārbaudīta vienā birojā divu nedēļu laikā, nevis daudzos birojos vairāku mēnešu garumā. Šie ierobežojumi norāda dabiskus nākamos soļus projekta attīstībai, kas ir uzskaitīti noslēguma nodaļā.

Bakalaura darbs izstrādāts **74** lapaspusēs un ietver **11** tabulas, **8** attēlus, **0** algoritmus, **12** programmu kodu izdrukas (listings), **2** pielikumus, kā arī literatūras sarakstu ar **20** avotiem.

Atslēgvārdi: starpplatformu mobilā izstrāde, Flutter, BLoC, Supabase, PostgreSQL, Row-Level Security, glabātās procedūras, divkāršā ieraksta grāmatvedība, naudas pārvedumi, ārvalstu valūtas, vairāku filiāļu operācijas, vairāku nomnieku arhitektūra, idempotence.

Abstract

In many countries of the former Soviet Union there are small offices that exchange one currency for another and send money abroad. A family in Uzbekistan, for example, may walk into such an office to receive money sent by a relative working in Russia, and the office, in turn, settles up with a partner office in Moscow once a week. These offices are often family businesses with branches in several cities and countries. They handle a lot of money every day, but they usually keep their records in a paper notebook or in an Excel spreadsheet. As a result, mistakes happen often, and a single missed line can lead to a loss of several thousand US dollars that takes weeks to find.

The reason mistakes happen is that the office really has to keep three kinds of accounts at the same time. The first is the money in its own cash drawer and on its own bank cards. The second is the running debt with every partner office abroad, and this debt has to be tracked in several currencies at once, because the same partner can owe the office both Russian roubles and Turkish liras at the same time. The third is the money belonging to regular customers, who often keep a small balance with the office in different currencies. Standard accounting programs mix all three of these into one number in one currency, and that hides exactly the things the office needs to see.

This Bachelor's Thesis describes a programme called *Ethnocount*, which the author designed and built to solve this real problem. The programme works on phones (Android, iPhone) and on computers (Windows, macOS, Linux) from the same code, so a branch can use the same tool whether the operator is sitting at a desk or running between clients with a phone in hand. The programme keeps the three kinds of accounts separate, the way they really are; it adds money up correctly even when the same partner has debts in three different currencies; and it makes sure that a transfer cannot be saved twice by accident, even if the operator taps the button twice on a bad internet connection. There are also rules about who can do what: the founder of the network sees everything, a director sees all branches, and a regular employee sees only his or her own branch and has to ask the director's permission before changing past records.

The author built the system, tested it carefully with many automatic tests, measured how fast it works on a normal Windows computer and on a middle-class Android phone, and

tried it for two weeks together with a real exchange office. During the trial the office kept both its old Excel spreadsheet and the new programme up to date, and the two were compared at the end of every week. The new programme did not introduce any mistakes of its own. On the contrary, it found three old mistakes in the spreadsheet that the operator had not noticed before.

The work still has limits, which are stated honestly. The programme does not yet download exchange rates from the internet on its own; the operator has to type them in. It does not let the operator save a transfer when there is no internet — in such a case the action is refused, because for money it is safer to refuse than to risk a wrong order later. And it was tested in one office over two weeks rather than in many offices over many months. These limits point to the natural next steps for the project, which are listed in the closing chapter.

The Bachelor Paper is written in **74** pages and contains **11** tables, **8** figures, **0** algorithms, **12** listings, **2** annexes, and a list of references with **20** sources.

Keywords: cross-platform mobile development, Flutter, BLoC, Supabase, PostgreSQL, Row-Level Security, stored procedures, double-entry accounting, money transfers, foreign exchange, multi-branch operations, multi-tenant architecture, idempotency.

Key words / Atslēgvārdi

Cross-platform mobile development

Flutter framework
BLoC state management
Supabase backend
PostgreSQL
Row-Level Security
Stored procedures (PL/pgSQL)
Double-entry bookkeeping
Treasury management
Money transfers
Foreign-exchange operations
Per-currency partner saldo
Multi-branch operations
Role-based access control
Approval workflow
Idempotent transactions
Clean Architecture
Design Science Research

Starpplatformu mobilā izstrāde

Flutter ietvars
BLoC stāvokļa pārvalde
Supabase aizmugure
PostgreSQL
Rindas līmeņa drošība
Glabātās procedūras (PL/pgSQL)
Divkāršā ieraksta grāmatvedība
Kases pārvaldība
Naudas pārvedumi
Ārvalstu valūtas operācijas
Partneru saldo pa valūtām
Vairāku filiāļu darbība
Lomu balstīta piekļuves kontrole
Apstiprināšanas darbplūsmas
Idempotentas transakcijas
Tīrā arhitektūra
Dizaina zinātnes pētījumi

Contents

Anotācija	1
Abstract	3
Key words / Atslēgvārdi	5
Introduction	12
1 Domain Context, Literature Review and Research Methodology	16
1.1 The Operational Picture of a Small Currency-Exchange Network	16
1.2 Double-Entry Bookkeeping in Software Systems	17
1.3 Multi-User Financial Applications: A Brief State of the Art	18
1.4 Cross-Platform Mobile Frameworks	19
1.5 Backend-as-a-Service Platforms	20
1.6 Clean Architecture, Domain-Driven Design and the BLoC Pattern	21
1.7 Research Methodology	22
1.8 Research Gap, Hypothesis and Success Criteria	24
1.9 Chapter Summary	24
2 System Analysis and Architecture Design	26
2.1 Functional and Non-Functional Requirements	26
2.1.1 Functional requirements	26
2.1.2 Non-functional requirements	27
2.2 Domain Model: Eight Entities Around Three Balance Domains	28
2.3 Architectural Pattern: Clean Architecture with BLoC	29
2.4 Database Schema and Domain Invariants	30
2.5 Multi-Tenant Security: Row-Level Security	32
2.6 The Director-Mediated Approval Workflow	33

2.7	Cross-Currency Quoting and Currency Tiers	34
2.8	Architectural Trade-offs	35
2.9	Chapter Summary	36
3	Implementation and Evaluation	37
3.1	Technology Stack and Project Structure	37
3.2	The Transfer Engine: Idempotency and Atomicity	39
3.3	Partner Operations and Per-Currency Saldo	42
3.4	The Approval Workflow in Code	43
3.5	Offline Behaviour and Real-Time Updates	45
3.6	UI/UX Design Language	45
3.7	Testing Strategy	46
3.8	Performance Measurements	47
3.9	User-Acceptance Trial	48
3.10	Limitations and Lessons Learned	49
3.11	Chapter Summary	50
	Conclusions and Recommendations	51
	Confirmation of the Research Tasks	51
	Validation of the Hypothesis	52
	Achievement of the Aim	52
	Scientific Contribution	53
	Practical and Theoretical Value	53
	Limitations and Future Work	54
	Closing Remark	55
	Literature	56
A	Database Schema (DDL Excerpt)	58
B	Deployment View and Transfer Sequence	63
B.1	Deployed Component View	63
B.2	Partner-Mediated Transfer Sequence	63
	Aplicinājums / Affirmation	65
	Pateicības / Acknowledgments	66

List of Figures

2.1	High-level entity–relationship sketch of the domain. Three balance domains (own funds, partner saldo, client wallet) are kept on separate axes and linked only by Transfer.	29
2.2	High-level architecture: dependencies always point inward toward the pure-Dart Domain layer.	29
2.3	Per-feature BLoC flow: Events flow toward the backend, States flow back to the screen.	30
2.4	The PendingApproval state machine: a request originates from an Accountant and terminates in one of three states.	34
3.1	Sketch of the Director’s approval screen: before-state and proposed after-state shown side by side, changed columns in bold.	44
3.2	Test pyramid of the platform.	46
B.1	Deployed component view of <i>Ethnocount</i> . Solid arrows are the write-path call flow; the dashed arrow is the Realtime change feed that refreshes the BLoCs after remote writes.	63
B.2	Sequence of a partner-mediated transfer from the operator’s tap to the server-side acknowledgement. The idempotency key (a UUID generated client-side in the form’s initialiser) travels with the request and is stored against a UNIQUE partial index on the server, so a duplicate request returns the original transfer ID rather than creating a second row.	64

List of Tables

1	Mapping of research tasks to thesis chapters	15
1.1	Representative existing systems compared against the niche of this thesis .	19
1.2	Cross-platform mobile frameworks considered at project start	20
2.1	Functional requirements (principal items)	26
2.2	Non-functional requirements	27
2.3	Currency tier map (excerpt)	35
2.4	Architectural trade-offs	35
3.1	Technology stack of the implementation	37
3.2	Principal integration-test scenarios (selection)	47
3.3	Median interaction latency on reference hardware ($n = 100$)	48
3.4	User-acceptance trial outcomes (two weeks)	48

List of Algorithms

Listings

2.1	The transfers table (principal columns)	31
2.2	Row-Level Security policies on transfers	32
3.1	Source tree (abridged)	38
3.2	Submitting a transfer (Dart, BLoC)	39
3.3	Stored procedure: create_transfer (abridged)	40
3.4	Per-currency saldo update (PL/pgSQL)	43
3.5	Pending approval: principal columns	44
A.1	Branches, users and branch accounts	58
A.2	Counterparties and per-currency saldo	59
A.3	Clients and per-client multi-currency wallet	60
A.4	Transfers, ledger entries and approvals	60
A.5	Audit log	62

Introduction

Topicality of the problem. Cross-border remittances handled by small currency-exchange offices form a sector that hardly anyone outside the trade thinks about, and yet it moves a non-trivial share of the labour-migrant economy of the post-Soviet space. The World Bank’s *Migration and Remittances Factbook* repeatedly placed countries such as Uzbekistan, Kyrgyzstan and Tajikistan among the world’s top recipients of remittances measured as a share of gross domestic product [14], and a sizeable portion of that flow does not pass through retail banks at all — it travels along chains of family-run exchange offices that settle with each other through informal partner networks. The operational tools available to those offices have remained, in practice, two: a paper journal and a Microsoft Excel spreadsheet. Off-the-shelf accounting software either presumes a trained bookkeeper at the keyboard or hides the multi-currency reality of the business behind a single *base-currency* ledger, which is precisely what the operator does not want to see. The result is a sector that is conspicuously under-served by software, despite handling sums measured in millions of US dollars per branch per year, and despite operating under a risk profile (cash, foreign-exchange, counterparty) that makes the absence of audit-grade records genuinely dangerous. This is the gap that the present thesis sets out to address.

Aim of the study. The aim of the work is to design, implement and evaluate a production-grade, cross-platform treasury and money-transfer platform — *Ethnocount* — tailored to small multi-branch currency-exchange networks, built on Flutter (client) and Supabase (backend), and demonstrably capable of replacing the spreadsheet-and-paper workflow for the daily operations of a representative operator.

Object of the study. The object of the study is the daily operational workflow of a small multi-branch currency-exchange network: the recording of own-branch cash and card movements, the issuing and receiving of cross-border money transfers through intermediary partner offices, the maintenance of per-currency running debt with each such partner, and the bookkeeping of recurring-client multi-currency wallets.

Subject of the study. The subject of the study is the design and implementation of a cross-platform information system that supports this workflow, expressed as a Flutter client (with the BLoC state-management pattern, GoRouter, get_it dependency injection and Hive local cache) on top of a Supabase backend (PostgreSQL 16, Row-Level Security, PL/pgSQL stored procedures and Realtime), under a Clean Architecture discipline and a role-based authorisation model with Creator, Director and Accountant roles.

Research problem. The research problem can be stated as follows: there is no off-the-shelf or published open-source system that simultaneously (i) preserves the three balance domains characteristic of the trade — own-branch funds, partner saldo, client wallet — without collapsing them into a single base-currency ledger; (ii) guarantees the atomic, idempotent creation of multi-table transfers under concurrent and intermittent network conditions; and (iii) implements a role-based authorisation model that lets branch accountants do their job while preventing them from modifying records outside their assigned branches except through an explicit Director-mediated approval workflow.

Tasks of the study. The aim is decomposed into the following tasks:

1. to review the relevant literature on double-entry bookkeeping in software, multi-user financial applications, cross-platform mobile frameworks and Backend-as-a-Service platforms, and to articulate the research gap in operational terms (Chapter 1);
2. to formalise the methodological framing of the work within Design Science Research and to describe the data sources used to elicit requirements (Chapter 1);
3. to derive the functional and non-functional requirements from interviews and observation of an operating exchange network, and to design the eight-entity domain model around the three-balance-domain principle (Chapter 2);
4. to specify the PostgreSQL schema and the Row-Level Security policy set, together with the Director-mediated approval workflow that protects multi-branch integrity (Chapter 2);
5. to implement the platform on Flutter and Supabase, including the idempotent atomic transfer engine, the per-currency partner-saldo subsystem and the dark-fintech design language (Chapter 3);

6. to evaluate the platform through unit, widget and integration tests, controlled performance measurements and a parallel-use trial with a real operator, and to honestly state the remaining limitations (Chapter 3).

Hypothesis. Combining Flutter (with the BLoC pattern) as the cross-platform user-interface layer and Supabase (PostgreSQL with Row-Level Security and stored procedures) as the backend permits the implementation of a three-balance-domain treasury platform that, on the workload of a small multi-branch exchange operator, is (i) free of the reconciliation errors observed in the prior spreadsheet workflow, (ii) responsive at sub-300-millisecond median interaction latency on commodity hardware, and (iii) maintainable by a single developer across a multi-month evolution cycle.

Research methods. The work uses systematic literature analysis, semi-structured stakeholder interviews and structured observation of the existing workflow, comparative analysis of candidate frameworks, system design under Clean Architecture and Domain-Driven Design vocabulary, software engineering on Flutter and PL/pgSQL, integration testing of stored procedures against a dedicated Supabase staging environment, instrumented performance measurement on Windows desktop and mid-range Android hardware, and a two-week parallel-use acceptance trial with the target operator.

Scientific novelty. Three contributions follow from the work. The first is an explicit, relationally expressed three-balance-domain accounting model with per-currency partner saldo — a model that, to the best of the author’s knowledge, has not been published in this shape and that resolves a real reconciliation pathology of the spreadsheet workflow. The second is a Director-mediated approval primitive that captures a before–after snapshot of every Accountant-initiated edit at request time, allowing the Director to compare states explicitly rather than rely on after-the-fact audit logs. The third is the demonstration that a single developer, using Flutter + BLoC + Supabase, can carry a treasury platform of non-trivial scope (15 tables, 66 migrations, approximately 70 000 lines of Dart) from concept to a working, operator-accepted system within the time-frame of a Bachelor’s thesis.

Approbation of the study. The complete source code of the platform is publicly released under a permissive open-source licence at <https://github.com/farruhsho/ethnocount>

for academic review and independent inspection of the architectural and engineering claims made in this thesis.

Task–chapter mapping. The six tasks listed above map onto the three substantive chapters of the thesis as shown in Table 1.

Table 1. Mapping of research tasks to thesis chapters

Task	Chapter
1. Review literature and articulate the research gap	Chapter 1
2. Frame the work within Design Science Research	Chapter 1
3. Elicit requirements and design the three-domain model	Chapter 2
4. Specify the PostgreSQL schema and the RLS policy set	Chapter 2
5. Implement the platform on Flutter and Supabase	Chapter 3
6. Evaluate the platform and state the remaining limitations	Chapter 3

1. Domain Context, Literature Review and Research Methodology

This chapter does three things. It paints the operational picture of the trade the platform is built for, so that the engineering decisions of the later chapters can be read in context. It then surveys the literature the work is built on, grouped around the four themes most relevant to the task at hand — double-entry bookkeeping inside software, multi-user financial applications, cross-platform mobile frameworks and Backend-as-a-Service platforms. It closes by stating the research methodology, naming the research gap and listing the criteria against which the work is judged.

1.1. The Operational Picture of a Small Currency-Exchange Network

Small currency-exchange offices in the post-Soviet region are usually family businesses with two to a dozen branches scattered across cities and countries. The branch is the working unit: it has a till, a card reader or two, a Wi-Fi connection that occasionally drops, and an operator who deals with walk-in customers in cash. The same office almost invariably also handles inbound and outbound money transfers — labour migrants sending earnings home, traders settling invoices, parents wiring tuition to children studying abroad. These transfers rarely travel through the SWIFT-and-correspondent-bank channels of retail finance. Instead, they hop along chains of informal *partner* offices in the destination country, settled later through a running ledger that is, in the field, often a piece of paper. The author's pre-thesis interviews touched a network with branches in Uzbekistan, Russia, Kazakhstan, Kyrgyzstan, Turkey, the United Arab Emirates and China, and the same shape repeated everywhere: one branch issues a transfer in cash; a partner abroad pays the recipient in another currency; the two offices each adjust their running saldo with each other, in the currency of the transfer, with a dealer-mode rate that captures the spread that is, in practice, the operator's profit.

A single transaction touches three accounting layers at once. The first is the operator's own till and card balances — the cash that physically lives in the branch and the funds available on the operator's settlement card. The second is the partner saldo: a running debt with the partner office, denominated, crucially, in several currencies simultaneously, because

the same partner may receive a payout in Russian roubles today and a payout in Turkish liras tomorrow. The third is the recurring client’s wallet, where a long-standing customer may keep a positive balance in dollars and a different positive balance in dirhams, drawn down as transfers are dispatched. Off-the-shelf software — 1C, QuickBooks, Wave — collapses these three layers into a single, single-currency ledger and absorbs the foreign-exchange differences into a generic *exchange rate variation* account. For a business whose entire revenue model is the spread between currencies, this collapse hides the very signal the operator needs to see.

The errors that emerge from this mismatch are not theoretical. During the requirements-elicitation phase of the project, the operator described several incidents in which a missing column in the spreadsheet, an unsigned partner adjustment, or a transfer recorded twice because of a flaky connection on the operator’s phone, produced a multi-thousand-US-dollar discrepancy that took weeks of forensic scrolling to reconcile. One of these incidents directly motivated the introduction of the idempotency key on transfer creation described in Chapter 3; another motivated the dedicated approval workflow on Accountant edits described in Chapter 2.

A second feature of the trade that shaped the design is its role structure. A typical network has a Creator (the founder, who owns the data and the credentials), one or more Directors (trusted managers who can see across all branches) and Accountants attached to specific branches (operators who run the day-to-day work but whose access to other branches’ data is restricted). Sensitive operations such as deleting a transfer or modifying a client record cannot be left at the Accountant’s discretion without compromising audit trails; equally, requiring the Director to log in for every correction would paralyse the workflow. The natural answer is a two-step protocol in which the Accountant *proposes* a change, the system captures the before-state, and the Director approves or rejects from a single screen. This is the approval workflow that section 2.6 formalises and that Chapter 3 implements as a PL/pgSQL RPC.

1.2. Double-Entry Bookkeeping in Software Systems

Double-entry bookkeeping is rarely cited in computer-science papers, but it is the silent backbone of nearly every accounting system in commercial use. The convention, codified in print by Luca Pacioli in 1494 [13] but considerably older in practice, records every economic event as two equal and opposite entries: a debit on one account, a credit on another,

summing to zero. The invariant is mechanical, which is precisely the point — a third party can detect arithmetic errors in a ledger by simple summation, without needing to understand the business it describes. The discipline survives in modern software, although the paired entries are usually generated automatically and hidden from the user interface.

In the software-engineering literature the relevance of double-entry has been argued repeatedly. Fowler, in *Patterns of Enterprise Application Architecture*, names the *Accounting Transaction* pattern as a near-universal building block for financial systems and warns explicitly against the naive shortcut of updating two balance rows in sequence without a wrapping transactional boundary [6]. The warning is borne out in practice: a sizeable share of the financial-software incidents documented in industrial post-mortems trace back to this exact failure mode, a row updated and the program crashed before the matching row was touched. Event-sourcing literature generalises the discipline further: each business event is recorded as an immutable fact in an append-only log, and balance views are derived as projections [5], [16]. The approach has the secondary virtue of producing an audit trail by construction.

The platform described in this thesis adopts a hybrid posture. The fundamental store is an append-only `ledger_entries` table in PostgreSQL, mirroring the event-sourcing tradition; balance rows are kept denormalised in dedicated tables for fast lookup. The atomicity of the dual write is enforced at the database layer through stored procedures that wrap both writes inside an implicit transaction, so that the schema preserves the double-entry invariant by construction rather than by trust in the application layer.

1.3. Multi-User Financial Applications: A Brief State of the Art

A review of consumer-facing mobile applications for shared finances exposes a sharp bifurcation. At one end sit informal expense-splitters — Splitwise being the canonical example — which prioritise frictionless peer-to-peer reconciliation among friends and intentionally avoid currency-level rigour [15]. At the other end sit small-business accounting suites such as Wave, Zoho Books and QuickBooks, which implement formal double-entry but presume a stable single-currency operating environment and a trained bookkeeper at the keyboard [10]. Between these two extremes the academic literature on cross-border remittance networks, in particular the World Bank *Migration and Remittances Factbook* series [14], documents the operational practices of intermediary settlement networks but does not produce a generalised software framework for them.

Enterprise treasury management systems — SAP Treasury, Kyriba [11] — do cover

the multi-currency multi-branch scenario natively, but they are priced and scoped for organisations several orders of magnitude larger than the target of this work, and their adoption curve is the territory of trained corporate treasury teams, not branch operators. The niche the present platform aims at therefore sits in a genuinely unoccupied region of the design space: small (two to a dozen branches), multi-currency, multi-tenant by role rather than by company, designed to be used by branch personnel under time pressure with walk-in clients waiting at the counter.

Table 1.1 compares the platforms reviewed in this section along the dimensions that mattered for the gap analysis: collaboration model, multi-currency support, partner-mediated transfer support, role-based authorisation and target operator profile.

Table 1.1. Representative existing systems compared against the niche of this thesis

System	Multi-cur.	Partner saldo	Role-based	Target
Splitwise	limited	—	—	peers
Wave / Zoho Books	limited	—	partial	small biz
QuickBooks	yes (collapsed)	—	yes	SMEs
1C: Accounting	yes (collapsed)	—	yes	trained accountant
SAP Treasury, Kyriba	yes	yes (corporate)	yes	enterprise
Ethnocount	yes (per-cur.)	yes (per-cur.)	yes	small exchange

The conclusion is that no surveyed system simultaneously preserves the three-balance-domain accounting model with per-currency partner saldo *and* targets the operational profile of a small exchange network with branch-level Accountants and a Director who oversees them. This is the gap the platform fills.

1.4. Cross-Platform Mobile Frameworks

Two waves dominate the cross-platform mobile space at the time of writing. The first, exemplified by React Native and Cordova, executes JavaScript through a bridge to native UI components: the developer writes once in a JavaScript dialect, and the framework instantiates platform-native widgets at runtime. The second, represented by Flutter, ships its own rendering engine and draws every pixel itself, so the application looks and behaves identically on every platform at the cost of bundling roughly twenty megabytes of engine code. The trade-offs are well-documented in the literature, including the often-cited comparative study by Heitkötter, Hanschke and Majchrzak [8], which is now slightly dated on specifics but still correct on the underlying axes: developer productivity, platform fidelity, accessibility

and the size of the runtime.

For the present project the choice fell on Flutter for three reasons that are pragmatic rather than ideological. The first is that the same Dart codebase targets Android, iOS and the three desktop platforms simultaneously, with Windows in particular non-negotiable: the network’s branches run on Windows desktops, and a web-only client was not acceptable. The second is that the Dart language is statically typed with sound null-safety, which catches a meaningful fraction of the bugs that would otherwise surface only in production. The third, more subjective but no less important for a single-developer project, is that Flutter’s hot reload — which redraws the user interface in under a second after a source change — makes the iterative-incremental development rhythm of section 1.7 tractable.

Table 1.2 compares the candidates considered.

Table 1.2. Cross-platform mobile frameworks considered at project start

Dimension	Flutter	React Native	Kotlin Multipl.
Android / iOS	stable	stable	stable
Windows desktop	stable	experimental	via shared logic only
macOS / Linux desktop	stable	experimental	limited
Type system	sound nulls	TypeScript optional	sound nulls
Hot reload	sub-second	sub-second	slower
Single-author velocity	high	medium	medium
Default look on Windows	matches design	off	native

1.5. Backend-as-a-Service Platforms

Backend-as-a-Service platforms emerged in the early 2010s as a way for small development teams to skip the operational burden of running their own application servers. Firebase pioneered the category, with a proprietary document store, opinionated security rules and a generous free tier; Supabase, launched in 2020 [3], set itself apart by exposing a full PostgreSQL database directly to the developer rather than a custom document layer. Supabase retains the queryability, the transactional guarantees and the strong type system of PostgreSQL, while wrapping them in managed services for authentication (GoTrue), automatic REST-and-RPC exposure (PostgREST), real-time logical replication over WebSockets (Realtime), object storage and edge functions.

The choice between the two for the present platform was, in the end, not difficult. The business semantics require that the dual write of a transfer (the source debit and the

destination credit) be atomic, and the natural way to express atomicity over multiple table writes is a database transaction wrapped in a stored procedure — not a custom security-rule DSL that cannot easily express cross-document constraints. Firebase Cloud Functions can wrap multiple Firestore writes in a single batched commit, but cross-document transactional behaviour in Firestore is intentionally bounded for scalability reasons [7], and the multi-table joins that the reporting screens of the platform rely on (transfers joined with branches joined with counterparties joined with currencies) are awkward to express against a document store. PostgreSQL handles those joins natively in single-millisecond time.

Brewer’s CAP theorem [2] reminds the architect that any distributed system must in the limit choose two out of consistency, availability and partition-tolerance. The platform under discussion prioritises consistency for write operations — a transfer either succeeds end-to-end or has no visible effect — and pays for that choice by refusing offline writes (section 3.5). The decision is appropriate to the domain: a missed read while the network is down is mildly inconvenient; a write that the operator believes succeeded but did not produce a row in transfers would be a financial incident.

1.6. Clean Architecture, Domain-Driven Design and the BLoC Pattern

The architectural vocabulary of the work is borrowed from two sources. Evans’ *Domain-Driven Design* [4] contributes the discipline of treating the business domain as a separately identifiable, pure-Dart core — entities, value objects, repositories as interfaces — around which the framework-bound layers orbit. Martin’s *Clean Architecture* [12] contributes the layering and the dependency rule: source dependencies point only inward, from outer concrete layers (presentation, data) toward the abstract domain core. The combination is not aesthetic but operational. It makes the domain entities unit-testable in isolation, without a Flutter test environment; it lets the backend (Supabase) be substituted with a mock implementation in tests without touching the BLoCs; and it ensures that when Flutter releases a breaking change in its widget library, the change touches only the presentation layer rather than rippling through the entire codebase.

The presentation layer uses the BLoC (Business Logic Component) pattern [1], in which every screen is driven by a small state machine whose inputs are typed *Events* and whose outputs are typed *States*. BLoC was preferred over Provider or Riverpod for two pragmatic reasons. The first is that the explicit Event–State model maps cleanly onto the transactional discipline that financial operations demand: every operator action corresponds

to an Event, every visible screen configuration corresponds to a State, and the developer cannot accidentally update part of the screen without producing the matching Event. The second is testability — events and states are pure values, exercised in isolation through the `bloc_test` package, without requiring a widget tree.

1.7. Research Methodology

The work is framed within Design Science Research (DSR) as formulated by Hevner, March, Park and Ram [9]. DSR posits that knowledge in engineering disciplines is produced through the construction of artifacts whose behaviour answers a defined research question. The artifact in this thesis is the Ethnocount platform itself; the knowledge contribution is the documented set of design decisions that allowed the three-balance-domain model to be expressed in a relational schema and operated atomically under the operational and authorisation constraints of the trade.

The DSR cycle used here proceeded through four iterations. In each iteration a focused sub-problem was identified — per-currency partner saldo in one iteration, the approval workflow in another, the balance-guard triggers in a third, AML/KYC foundations in a fourth — a candidate design was implemented, the design was evaluated against the criteria stated in the hypothesis, and the lessons learned informed the next iteration. The iterations were not strictly sequential: several earlier decisions were revisited when later iterations exposed their limits. Migration 047 of the production schema, for example, replaced a USD-collapsed partner saldo with the per-currency JSONB model only after iteration three had shown the original collapse to hide real exposure when the same partner had simultaneous debts in three currencies (the trigger event was described by the operator during an interview between iterations two and three).

The day-to-day work inside each iteration followed an iterative-incremental engineering rhythm closer in spirit to disciplined Agile than to ceremonial Scrum. A single developer worked through a backlog organised around vertical slices: every slice delivered a coherent feature end-to-end, from the migration through the stored procedure and the BLoC to the screen the operator would actually use. Slices were kept small enough to keep the trunk continuously deployable, and database migrations were numbered, idempotent SQL files that could be replayed in any environment. Source control was Git; every commit referenced the slice it implemented. Code review was self-administered with the help of the Dart analyser configured for strong null-safety and the Supabase advisor for the database layer. A short

handoff document was kept current at all times in case the project needed to be passed to another developer — a discipline that, although not used in anger, was helpful in surfacing implicit assumptions.

Data sources. The functional requirements were elicited from three complementary sources. The first was a series of semi-structured interviews with the operator running the existing spreadsheet workflow, covering the activities of a typical day, the kinds of errors the operator had experienced and the data items currently kept in the operator’s head rather than in any artifact. The second source was structured observation: the author sat with the operator across four working sessions, watching the spreadsheet being filled in real time and noting moments of hesitation, rework or branched logic. The third source was the existing spreadsheet itself, treated as a tacit specification — column names, conditional formats and hand-written notes in the margin were all carefully reverse-engineered into entity attributes.

Evaluation methods. The evaluation, reported in Chapter 3, combines four complementary methods. Automated unit and widget tests verify the correctness of the pure-Dart logic and the rendering of the most safety-critical screens under controlled state. Integration tests against a dedicated Supabase staging project verify the behaviour of stored procedures and Row-Level Security policies, including the atomicity properties claimed in the hypothesis. Performance measurements are taken on a representative Windows desktop and a mid-range Android device, instrumented through chronometers around the most frequent user actions. Finally, a User Acceptance Test was conducted with the operator over two weeks of parallel use, during which both the spreadsheet and the platform were updated with the same operational data and reconciled at the end of each week.

Ethical considerations. The interviews and the acceptance trial involved a single business and were conducted under explicit consent. No client-identifying data is included in this thesis; where examples appear, names, phone numbers and account numbers are illustrative placeholders. The codebase contains no hard-coded credentials, and the staging Supabase project used for evaluation was destroyed at the conclusion of the trial. The thesis acknowledges the regulatory complexity of remittance services in some jurisdictions and explicitly limits its scope to the operational dimension of the business rather than its legal one.

1.8. Research Gap, Hypothesis and Success Criteria

Synthesising the literature reviewed in this chapter with the operational picture of section 1.1, the research gap can be stated precisely: there is no published or off-the-shelf system that simultaneously (i) preserves the three-balance-domain accounting model with per-currency partner saldo, (ii) enforces atomicity and idempotency of multi-table transfer creation through database-level stored procedures rather than application-level checks, and (iii) combines Row-Level Security with a Director-mediated approval workflow that lets branch Accountants do their work while protecting multi-branch integrity. The hypothesis stated in the Introduction is that a Flutter + BLoC client on a Supabase backend is a sufficient technology stack to close this gap within the engineering envelope of a single bachelor-level developer.

The success criteria against which the rest of the thesis is judged are three. *Correctness* requires that the platform exhibit zero platform-introduced reconciliation errors during the two-week parallel-use trial. *Responsiveness* requires sub-300-millisecond median interaction latency on the reference hardware. *Maintainability* requires that the platform be carried, by a single developer, through a multi-month evolution cycle without irreparable drift between the client and the server schema — operationalised in this work as: every commit must land on a green build, and every schema change must arrive as a numbered, idempotent migration file.

1.9. Chapter Summary

The chapter has done four things. It described the operational picture of the small currency-exchange networks the platform is built for, with its three accounting layers, its Creator/Director/Accountant role structure and its tolerance for hand-held devices on flaky networks. It surveyed the literature relevant to the engineering task — double-entry bookkeeping in software, multi-user financial applications, cross-platform frameworks, Backend-as-a-Service platforms, and the architectural vocabulary of Clean Architecture, Domain-Driven Design and the BLoC pattern. It framed the work within Design Science Research, described the iterative-incremental development rhythm and the four data sources used for requirements elicitation. And it formalised the research gap and the three success criteria against which the rest of the work is assessed.

Conclusions for Chapter 1. No surveyed system simultaneously preserves the three-balance-

domain accounting model with per-currency partner saldo, enforces atomicity through database stored procedures, and combines Row-Level Security with a Director-mediated approval workflow. The hypothesis is that the combination of Flutter (with BLoC) and Supabase (PostgreSQL with RLS and stored procedures) is sufficient to close this gap inside the engineering envelope of a single bachelor-level developer. The next chapter derives the requirements, designs the domain model and specifies the database schema and the RLS policy set against which that hypothesis will be tested.

2. System Analysis and Architecture Design

The previous chapter framed the problem and the research methodology. The present chapter turns the abstract gap into a system. It begins with the catalogue of functional and non-functional requirements derived from the elicitation activities of section 1.7; it proceeds through the domain model, the Clean Architecture pattern, the PostgreSQL schema and the Row-Level Security policy set; and it closes with the design of the Director-mediated approval workflow that turns out to be the central authorisation primitive of the platform.

2.1. Functional and Non-Functional Requirements

2.1.1. Functional requirements

The functional requirements were derived from the three data sources of section 1.7 — the operator interviews, the four working-session observations and the existing spreadsheet treated as a tacit specification — and refined across the four iterations of the DSR cycle. Table 2.1 lists the principal requirements with stable identifiers that are referenced from the implementation chapter.

Table 2.1. Functional requirements (principal items)

ID	Requirement
FR-01	The system shall authenticate users by email and password and load their role and assigned-branches list immediately after sign-in.
FR-02	The system shall present, on a dashboard, the per-currency balances of every branch the user is authorised to see.
FR-03	The system shall let an authorised user create a money transfer in one of three modes: own-branch only; partner-mediated (the transfer travels through an intermediary partner); and dealer mode (with explicit buy/sell rates for the spread).
FR-04	The system shall accept an idempotency key on every transfer-creation request and reject any duplicate request carrying the same key.
FR-05	The system shall maintain a per-currency running saldo with every intermediary partner, updated atomically whenever a partner-mediated transfer is created or detached.
FR-06	The system shall maintain a per-client multi-currency wallet (ClientBalance) and shall record every wallet movement as an immutable ledger entry.

ID	Requirement
FR-07	The system shall allow the Creator and the Director to administer the catalogues of branches, branch accounts, partners, clients, commissions and exchange rates.
FR-08	The system shall restrict an Accountant's visibility to the branches listed in the user's assigned-branch-ids attribute.
FR-09	The system shall route Accountant-initiated modifications of transfers, clients or accounts through a pending-approvals workflow that captures a before-snapshot of the affected row.
FR-10	The system shall present the Director with a single screen on which the before- and after- states of each pending approval are visible side by side, and on which the Director can approve or reject.
FR-11	The system shall send real-time notifications to relevant users when a transfer is created, modified, deleted or approved/rejected.
FR-12	The system shall log every modification of a sensitive entity (transfer, client, account, partner) to an audit trail that records the actor, the timestamp and the before/after states.
FR-13	The system shall produce per-currency reconciliation reports between the operator's book and the partner's book over a chosen period.

2.1.2. Non-functional requirements

The non-functional requirements (Table 2.2) translate the hypothesis of section 1.8 into measurable engineering targets. They are referenced again in the evaluation section of Chapter 3.

Table 2.2. Non-functional requirements

ID	Property	Target
NFR-01	Median interaction latency	≤ 300 ms on reference hardware
NFR-02	Cross-tenant data leakage	Zero rows under integration tests
NFR-03	Atomicity of transfer creation	All-or-nothing across all affected tables
NFR-04	Idempotency of write operations	Server rejects duplicate idempotency key
NFR-05	Cross-platform reach	Android, iOS, Windows, macOS, Linux
NFR-06	Single-developer maintainability	Continuously deployable trunk; numbered idempotent migrations
NFR-07	Audit completeness	Every sensitive change recorded with actor and before/after states

2.2. Domain Model: Eight Entities Around Three Balance Domains

The central architectural commitment of the platform is to express the three accounting layers as three distinct sets of entities, linked by atomic stored procedures rather than collapsed into a common base-currency ledger. Eight entities carry this model:

Branch. A physical or notional office. Identified by a UUID. Carries a name, a city, a country, a default currency and a creator-owned flag.

BranchAccount. A till or settlement card associated with a branch. Carries a denominated currency and a running balance. The canonical store for the operator's own funds.

Counterparty. An intermediary partner office, typically abroad. Carries a name, a contact channel and the central attribute of the platform: a `saldo_by_currency` JSONB object keyed by ISO currency code.

CounterpartyTransaction. An immutable event that mutates a counterparty's saldo: a `paid_for_us` when the partner has paid out a transfer on the operator's behalf, a `paid_for_them` when the operator has paid out a transfer on the partner's behalf, a `settlement` when one party has wired cash to the other to zero out the running saldo.

Client. A recurring customer. Carries name, identification (passport or equivalent), KYC notes, and a link to a `ClientBalance`.

ClientBalance. A multi-currency wallet owned by a client. The wallet's per-currency totals are derived from the immutable `ClientTransaction` stream.

Transfer. The principal entity of the platform. Carries a source account, a destination account or partner, source and destination amounts, source and destination currencies, an exchange rate, a status and an idempotency key. Optionally linked to a counterparty through the `via_counterparty_id` attribute.

LedgerEntry. An immutable, append-only fact about a value movement. Every `Transfer` produces one or more `LedgerEntries`; so does every settlement; so does every wallet movement.

A ninth supporting entity — `PendingApproval` — materialises the Director-mediated approval workflow. It carries the proposed change, a snapshot of the affected row at the moment of request, and an outcome (approved, rejected, withdrawn).

Figure 2.1 sketches the relationships and cardinalities at a high level. The principal observation is that the three balance domains do not share rows: a `BranchAccount` row is never reused as a `ClientBalance`, even though the underlying schema could in principle express it; the conceptual separation is enforced by the foreign-key directions and by the stored-procedure interfaces.

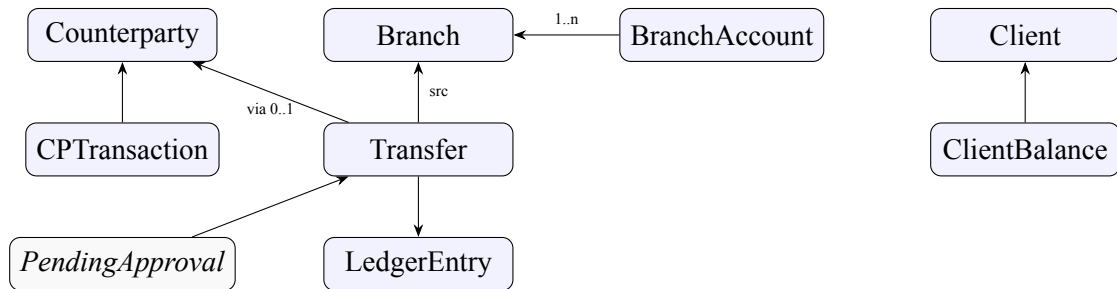


Figure 2.1. High-level entity-relationship sketch of the domain. Three balance domains (own funds, partner saldo, client wallet) are kept on separate axes and linked only by Transfer.

2.3. Architectural Pattern: Clean Architecture with BLoC

The application follows a Clean-Architecture variant adapted to Flutter, structured as four layers (Figure 2.2). The *domain* layer holds pure-Dart entities, value objects (`Money`, `CurrencyCode`, `ExchangeRate`) and repository interfaces; it has no dependency on Flutter, Supabase or Hive. The *data* layer contains repository implementations, remote data sources backed by the Supabase Dart client, and DTO serialisers. The *presentation* layer hosts BLoCs and screens. The *core* layer collects cross-cutting helpers — the `get_it` container that wires the dependency graph, the GoRouter route table, the AppIcons facade that abstracts away the icon-pack identifiers, the currency-tier utility, the network connectivity probe.

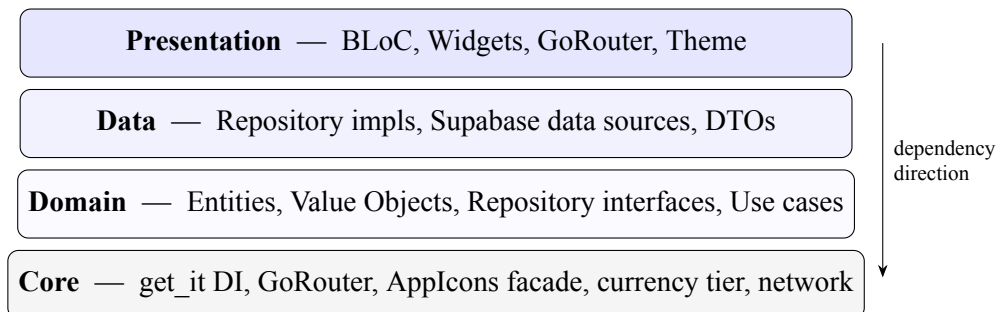


Figure 2.2. High-level architecture: dependencies always point inward toward the pure-Dart Domain layer.

The BLoC pattern was chosen, as section 1.6 discussed, because the explicit Event-
State model maps cleanly onto the transactional discipline of financial operations. Within

the presentation layer, an additional pattern emerged across iterations: *shared widgets* for cross-screen blocks (the dealer-rates block, the partner-saldo preview, the currency picker), grouped under `lib/presentation/common/widgets/` and reused by the transfer-creation screen, the partner-card screen and the attach dialog. This avoided the divergence that would otherwise creep in between two surfaces that conceptually do the same thing.

Figure 2.3 sketches the BLoC flow for the Transfers feature: the screen dispatches Events to a TransfersBloc, the BLoC consults the TransferRepository (which in turn consults the SupabaseTransferDataSource and the local cache), and the BLoC emits a sequence of States that the screen renders.

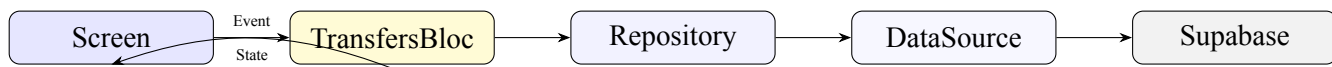


Figure 2.3. Per-feature BLoC flow: Events flow toward the backend, States flow back to the screen.

2.4. Database Schema and Domain Invariants

The PostgreSQL schema is currently expressed across 15 tables and 66 numbered, idempotent migrations. The principal tables are `users`, `branches`, `branch_accounts`, `counterparties`, `counterparty_transactions`, `clients`, `client_balances`, `client_transactions`, `transfers`, `ledger_entries`, `commissions`, `exchange_rates`, `pending_approvals`, `notifications` and `audit_log`.

Three domain invariants govern the schema:

- I_1 (**double-entry**). Every transfer produces exactly two matching ledger entries: a credit on the source account in the source currency, a debit on the destination account in the destination currency. Both entries are inserted inside the same database transaction, or neither is.
- I_2 (**tenant isolation**). For every row r in a tenant-scoped table T (`transfers`, `clients`, `accounts`, `partners`, etc.), r is visible to user u if and only if $r.branch_id$ is contained in the user’s branch set — itself the union of all branches for Creator/Director and the user’s `assigned_branch_ids` for Accountant.
- I_3 (**idempotency**). Every transfer-creation request carries a client-supplied UUID, stored in the `idempotency_key` column. A UNIQUE partial index on this column rejects any duplicate request at the database layer.

Listing 2.1 shows the principal columns of the transfers table. The partial index on idempotency_key excludes NULL so that legacy rows from before the introduction of the key are not in scope.

Listing 2.1. The transfers table (principal columns)

```
1 CREATE TABLE transfers (  
2   id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   branch_id         UUID NOT NULL REFERENCES branches(id),  
4   via_cp_id         UUID REFERENCES counterparties(id),  
5   src_account_id    UUID NOT NULL REFERENCES branch_accounts(id),  
6   dst_account_id    UUID REFERENCES branch_accounts(id),  
7   src_currency      TEXT NOT NULL,  
8   dst_currency      TEXT NOT NULL,  
9   src_amount        NUMERIC(20,4) NOT NULL CHECK (src_amount > 0),  
10  dst_amount        NUMERIC(20,4) NOT NULL CHECK (dst_amount > 0),  
11  rate              NUMERIC(20,10) NOT NULL,  
12  status            TEXT NOT NULL DEFAULT 'delivered',  
13  idempotency_key   UUID,  
14  created_by        UUID NOT NULL REFERENCES users(id),  
15  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
16  amendment_history JSONB NOT NULL DEFAULT '[]'::jsonb  
17 );  
18  
19 CREATE UNIQUE INDEX transfers_idem_uk  
20   ON transfers(idempotency_key)  
21   WHERE idempotency_key IS NOT NULL;
```

A sanity check that turned out to be operationally important guards the client_transactions amount: a CHECK constraint rejects amounts outside the half-open interval $(0, 10^{12}]$, which excludes both negative values and pathological large ones. The constraint was introduced after a real incident in which a malformed input produced a row with an amount of $\approx 10^{61}$ US dollars and broke the rendering layer; the constraint now treats the rendering crash as a database-level rejection.

2.5. Multi-Tenant Security: Row-Level Security

The security model is enforced at the database layer through PostgreSQL Row-Level Security. Three roles are recognised. A *Creator* (the founder) holds unconditional read and write access to every row in the project. A *Director* sees data from all branches and can administer the catalogues of partners, accounts, commissions and exchange rates. An *Accountant* sees only the branches listed in the user's `assigned_branch_ids` attribute and can write only against those branches; modifications to existing rows are channelled through the approval workflow of section 2.6.

Listing 2.2 shows the principal RLS policies on the `transfers` table. The pattern — a `SELECT` policy parameterised on the user's branch set and a separate `WITH CHECK` policy on the role — is the canonical shape used across all tenant tables in the schema.

Listing 2.2. Row-Level Security policies on transfers

```
1 ALTER TABLE transfers ENABLE ROW LEVEL SECURITY;
2
3 CREATE POLICY transfers_select_visible
4   ON transfers
5   FOR SELECT
6   USING (
7     (SELECT role FROM users WHERE id = auth.uid())
8     IN ('creator', 'director')
9   OR branch_id = ANY(
10    (SELECT assigned_branch_ids FROM users WHERE id = auth.uid
11      ()))
12   );
13
14 CREATE POLICY transfers_insert_branch_authorised
15   ON transfers
16   FOR INSERT
17   WITH CHECK (
18     branch_id = ANY(
19       (SELECT assigned_branch_ids FROM users WHERE id = auth.uid())
20     )
```

```

21      OR (SELECT role FROM users WHERE id = auth.uid())
22          IN ('creator', 'director')
23  );
24
25  -- Direct UPDATE and DELETE are not granted to Accountants;
26  -- they go through the approval workflow (see Section 2.7).

```

The threat model the policies have to defend against is straightforward but deserves stating. A malicious or simply confused client may forge any `branch_id` in any request, replay any operation, attempt to mutate a row not belonging to its branch set, or read rows belonging to other branches. Under the RLS regime, none of these can succeed by accident: the PostgreSQL query planner rewrites every query to add the policy predicate, so a `SELECT` against a transfer outside the user’s branch set returns zero rows rather than an authorisation error, and an `INSERT` or `UPDATE` that violates the `WITH CHECK` predicate raises an error before any row is touched. Cross-tenant leakage thus becomes a structural property of the query plan, not a runtime check that the application layer might forget.

2.6. The Director-Mediated Approval Workflow

The approval workflow is, in the author’s judgement, the most novel architectural primitive of the platform. The problem it solves is concrete. An Accountant attached to the Tashkent branch realises, while reconciling the day’s takings, that yesterday’s transfer ID T-2156 had the wrong destination currency. Letting the Accountant simply correct the row would compromise the audit trail and would tempt the Director to monitor every Accountant action defensively. Requiring the Director to log in personally for every correction would, in turn, paralyse the workflow. The approval workflow is the synthesis: the Accountant *proposes* a correction, the system captures the row’s current state as a snapshot, the Director receives a notification and approves or rejects from a dedicated screen on which the before-state and the after-state are visible side by side.

Figure 2.4 sketches the state machine. The transitions are guarded by separate stored procedures (`request_approval`, `approve`, `reject`, `withdraw`), each of which checks the actor’s role through the RLS-aware `auth.uid()` hook.

The snapshot captured at request time is the part that gives the workflow its forensic value. Without it, the Director would have to reason about the proposed change in the ab-

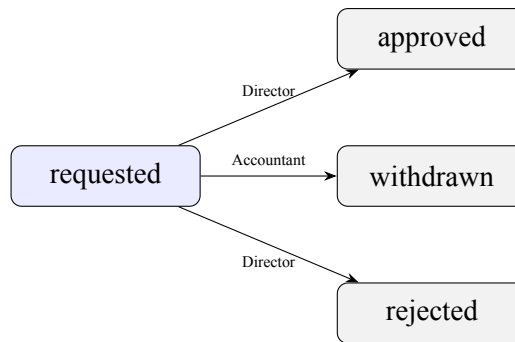


Figure 2.4. The PendingApproval state machine: a request originates from an Accountant and terminates in one of three states.

struct; with it, the Director sees the two states explicitly and the after-state highlights only the columns that the Accountant proposed to change. The snapshot is stored as a JSONB column on `pending_approvals`, which keeps the schema flexible across different sensitive tables (transfers, clients, accounts) without requiring a fresh column per entity.

A consequence of channeling all sensitive Accountant edits through this workflow is that the audit log effectively writes itself: the `audit_log` table receives an entry for every approved request, with actor, timestamp, the proposed change and the originally captured before-state. The author found, over the course of the project, that this side-effect saved more debugging time than any other single design choice.

2.7. Cross-Currency Quoting and Currency Tiers

A small but consequential design question is the direction in which the operator should enter exchange rates. In a pair such as Uzbek som / Russian rouble, should the operator type $1\ RUB = X\ UZS$ or $1\ UZS = X\ RUB$? The choice matters because it changes the precision of the entered number and the orientation of every preview on the screen. The platform answers the question by assigning every supported currency a numeric *tier*: the lower the tier, the stronger the currency. When two currencies are compared, the stronger one (lower tier) becomes the base and the weaker one becomes the quoted unit. Table 2.3 reproduces the principal tiers in use.

The `quotePair(a, b)` helper, exposed as a pure-Dart function in `lib/core/utils/currency_`, always returns the stronger side first; the rest of the user interface — transfer form, partner-attach dialog, dealer-rates block — relies on this single source of truth so that the operator never has to mentally invert a quote. An equivalent SQL view is used inside stored procedures for the rare cases where the server has to display a rate without re-entering it.

Table 2.3. Currency tier map (excerpt)

Tier	Code	Currency
1	USD	US dollar
1	EUR	Euro
1	GBP	Pound sterling
2	AED	UAE dirham
2	CNY	Chinese yuan
3	TRY	Turkish lira
3	RUB	Russian rouble
4	KZT	Kazakh tenge
4	KGS	Kyrgyz som
5	UZS	Uzbek som

2.8. Architectural Trade-offs

The principal architectural decisions of the platform are summarised in Table 2.4, with the chief alternative considered and the rationale for the chosen option. Several of these decisions were re-litigated mid-project; the column *Iteration* records the iteration of the DSR cycle in which the decision settled.

Table 2.4. Architectural trade-offs

Decision	Alternative	Rationale	Iter.
Flutter + Dart	React Native, KMP	Same Dart code on 5 platforms; sound nulls; sub-second hot reload.	1
Supabase (PostgreSQL + RLS)	Firebase	RLS expressivity; multi-table transactions; PL/pgSQL.	1
BLoC + Cubit	Provider, Riverpod	Explicit Event–State model; testability; mature ecosystem.	1
Per-currency partner saldo (JSONB)	USD-collapsed saldo	Hides genuine exposure when partner has debts in 3 currencies.	3
Director-mediated approval workflow	Direct accountant writes	Preserves audit trail; protects multi-branch integrity.	2
Idempotency key on transfer creation	Server-issued ETag	Works through retries; UUID generated client-side; zero RTT.	1
Refuse offline writes	Outbox + replay	Financial semantics make refuse safer than reorder.	2
Sanity caps on client amounts	Trust the client	A real input produced 10 ⁶¹ USD before the cap.	3

Decision	Alternative	Rationale	Iter.
Snapshot at request time	Snapshot at approval time	Captures Accountant's intent against actual state at the time.	2
Numbered idempotent migrations	Hand-rolled changes	DB Replayable across staging and production; single source of truth.	1
AppIcons facade over lucide_icons_flutter	Direct icon use	Prevents accidental Material icons; one place to swap packs.	1

2.9. Chapter Summary

The chapter has done five things. It catalogued the functional and non-functional requirements derived from interviews, observation and the existing spreadsheet workflow. It described the eight-entity domain model and explained why the three balance domains are kept on separate axes. It situated the application inside a Clean-Architecture variant with BLoC at the presentation layer and core utilities collected in their own cross-cutting layer. It specified the PostgreSQL schema, the three domain invariants and the Row-Level Security policy set that enforces tenant isolation by query-plan rewriting. And it designed the Director-mediated approval workflow that protects multi-branch integrity while preserving Accountant usability.

Conclusions for Chapter 2. The three-balance-domain accounting model can be expressed in a relational schema, with the per-currency partner saldo encoded as a JSONB attribute keyed by ISO currency code. The atomicity, idempotency and tenant-isolation invariants (I_1 , I_2 , I_3) can be enforced at the database layer through transactions, UNIQUE partial indexes and Row-Level Security policies. The Director-mediated approval workflow gives the operator a structured way to channel sensitive Accountant edits while preserving the audit trail. Chapter 3 turns this design into running code and reports the empirical evaluation that judges it against the success criteria of section 1.8.

3. Implementation and Evaluation

This chapter turns the design of Chapter 2 into running code and reports the empirical evaluation. It begins with the technology stack and the layout of the project tree, drills into the four engineering subsystems that carry most of the platform’s risk — the transfer engine, the per-currency partner-saldo subsystem, the offline behaviour and the design system — and closes with the testing strategy, the performance measurements, the user-acceptance trial and an honest account of the limitations that remain.

3.1. Technology Stack and Project Structure

The implementation rests on a deliberately narrow stack. On the client the platform uses Flutter 3 with Dart 3, the BLoC library (`flutter_bloc` 9.x) for state management, `get_it` with `injectable` for dependency injection, `go_router` for navigation, `hive` for a small local cache (used only for read-side resilience — see section 3.5), `trina_grid` for desktop data grids, and `lucide_icons_flutter` wrapped by an internal `AppIcons` facade. On the server the platform uses Supabase — managed PostgreSQL 16, GoTrue for authentication, PostgREST and a thin RPC layer for direct stored-procedure calls, Realtime for logical-replication subscriptions, and Storage. The complete dependency list is reproduced in Table 3.1.

Table 3.1. Technology stack of the implementation

Layer	Technology	Version
Language	Dart	3.x
UI framework	Flutter	3.x stable
State management	<code>flutter_bloc</code>	9.x
Dependency injection	<code>get_it</code> + <code>injectable</code>	8.x / 2.x
Routing	<code>go_router</code>	15.x
Local cache	<code>hive</code> + <code>hive_flutter</code>	2.x / 1.x
Code generation	<code>freezed</code> + <code>json_serializable</code>	3.x / 6.x
Iconography	<code>lucide_icons_flutter</code> + <code>AppIcons</code> facade	0.5
Desktop grids	<code>trina_grid</code>	1.x
Charts	<code>fl_chart</code>	0.70
Backend	Supabase (PostgreSQL 16 + Realtime)	latest stable
Tests	<code>flutter_test</code> , <code>bloc_test</code> , <code>mocktail</code>	latest

The source tree (Listing 3.1, abridged) follows the Clean-Architecture layering of

section 2.3. Each top-level directory under `lib/` maps to a single layer; each presentation feature has its own folder with three sub-folders for BLoC, pages and widgets.

Listing 3.1. Source tree (abridged)

```
1 lib/
2   app.dart                # MaterialApp + theming
3   main.dart               # Bootstrap, DI, Supabase init
4   core/
5     constants/            # Currency tier map, role enum
6     di/                   # get_it configuration
7     icons/                # AppIcons facade
8     network/              # Connectivity probe
9     routing/              # GoRouter table
10    services/              # Cross-cutting services
11    supabase/              # Typed Supabase client
12    theme/                 # Dark-fintech palette
13    utils/                 # Money, currency helpers
14    domain/
15      entities/            # Pure-Dart entities
16      repositories/        # Abstract interfaces
17      services/            # Pure-Dart domain services
18      usecases/            # Use case classes
19    data/
20      datasources/remote/  # SupabaseTransfersDataSource, ...
21      repositories/        # Repository implementations
22      services/            # Data-layer services
23    presentation/
24      auth/                # Authentication
25      dashboard/           # Branch / per-currency totals
26      transfers/           # Transfer creation, list, attach
27      counterparties/      # Partner cards, attach flows
28      clients/             # Client list, wallet
29      branches/            # Branch admin
30      users/               # User admin
31      reports/             # Per-currency reconciliation
```



```

32     analytics/                # Charts
33     notifications/            # Real-time inbox
34     admin/                    # Director approval queue
35     settings/                 # Preferences
36     common/widgets/           # Shared widgets
37 supabase/migrations/         # 66 numbered .sql files

```

The dependency-injection container is initialised once at startup. It registers the Supabase client as a singleton, then registers a remote data source per domain area, then a repository per area on top of it. The BLoC layer pulls its repositories from this container, which keeps every BLoC ignorant of how the data layer is wired.

3.2. The Transfer Engine: Idempotency and Atomicity

The transfer creation procedure is the most safety-critical part of the system. The client side issues a transfer-creation request that carries an idempotency key (a UUID generated in the initialiser of the form widget, before the user has a chance to tap *Submit* twice). Listing 3.2 shows the BLoC code path.

Listing 3.2. Submitting a transfer (Dart, BLoC)

```

1 class TransfersBloc extends Bloc<TransfersEvent, TransfersState> {
2   TransfersBloc(this._repo) : super(const TransfersState.idle()) {
3     on<SubmitTransfer>(_onSubmit);
4   }
5
6   Future<void> _onSubmit(
7     SubmitTransfer event, Emitter<TransfersState> emit) async {
8     emit(const TransfersState.submitting());
9     final result = await _repo.createTransfer(
10      branchId: event.branchId,
11      srcAccountId: event.srcAccountId,
12      dstAccountId: event.dstAccountId,
13      srcAmount: event.srcAmount,
14      dstAmount: event.dstAmount,
15      rate: event.rate,
16      viaCounterpartyId: event.viaCounterpartyId,

```

```

17     idempotencyKey: event.idempotencyKey, // UUID v4 from form
        init
18 );
19 result.fold(
20     (failure) => emit(TransfersState.error(failure.
        toReadableMessage())),
21     (transfer) => emit(TransfersState.created(transfer)),
22 );
23 }
24 }

```

The server side wraps the entire creation procedure in a single PL/pgSQL stored function with SECURITY DEFINER, which guarantees that all subordinate operations execute inside the same implicit transaction. Listing 3.3 shows an abridged form of the procedure. The function validates the inputs, picks the next sequential transaction code, inserts the row into transfers, updates branch_accounts, inserts the matching ledger entries, optionally records a counterparty transaction and a commission row, and writes an audit-log entry. Any failure raises an exception, which rolls back the whole transaction.

Listing 3.3. Stored procedure: create_transfer (abridged)

```

1 CREATE OR REPLACE FUNCTION create_transfer(
2     p_idempotency_key    UUID,
3     p_branch_id          UUID,
4     p_src_account_id     UUID,
5     p_dst_account_id     UUID,
6     p_via_cp_id          UUID,
7     p_src_amount         NUMERIC,
8     p_dst_amount         NUMERIC,
9     p_rate               NUMERIC
10 ) RETURNS UUID
11 LANGUAGE plpgsql AS $$
12 DECLARE
13     v_existing UUID;
14     v_transfer  UUID;
15     v_actor     UUID := auth.uid();

```

```

16 BEGIN
17   -- Short-circuit on duplicate
18   SELECT id INTO v_existing FROM transfers
19     WHERE idempotency_key = p_idempotency_key;
20   IF FOUND THEN RETURN v_existing; END IF;
21
22   -- Role and branch authorisation
23   PERFORM assert_can_write_branch(v_actor, p_branch_id);
24
25   -- Atomic value movement
26   INSERT INTO transfers (...) VALUES (...) RETURNING id INTO
       v_transfer;
27   UPDATE branch_accounts SET balance = balance - p_src_amount
28     WHERE id = p_src_account_id;
29   IF p_dst_account_id IS NOT NULL THEN
30     UPDATE branch_accounts SET balance = balance + p_dst_amount
31       WHERE id = p_dst_account_id;
32   END IF;
33   INSERT INTO ledger_entries (...) VALUES (...);   -- credit on
       source
34   INSERT INTO ledger_entries (...) VALUES (...);   -- debit on
       destination
35
36   -- Partner-mediated branch: adjust per-currency saldo
37   IF p_via_cp_id IS NOT NULL THEN
38     INSERT INTO counterparty_transactions (...) VALUES (...);
39     PERFORM bump_saldo(p_via_cp_id,
40                       p_dst_currency,
41                       p_dst_amount);
42   END IF;
43
44   PERFORM audit('transfer', v_transfer, 'create', v_actor);
45   RETURN v_transfer;
46 END $$;

```

The properties the function gives in exchange for its size are worth stating explicitly. Atomicity is structural: every subordinate operation executes inside the same transaction, so either all succeed or none take effect. Idempotency is structural too: the UNIQUE partial index on `idempotency_key` ensures that a second call with the same key short-circuits at the first lookup and returns the original transfer ID. Authorisation is double-belt-and-braces: even though Row-Level Security would block an unauthorised client, the `assert_can_write_branch` call raises a loud exception, which surfaces in monitoring and is easier to debug than silent zero-row writes. The complexity per call is $O(1)$: a handful of indexed lookups and a handful of inserts.

3.3. Partner Operations and Per-Currency Saldo

Partner-mediated transfers are exposed in three places in the user interface. The first is the regular transfer-creation form, which has an optional *via partner* field; selecting a partner here switches the form into partner-mediated mode and adds the partner-saldo preview block. The second is an *attach* dialog reached from an existing transfer in the list, which converts an already-created own-branch transfer into a partner-mediated one without altering the value movement on the operator's side. The third is the symmetric *attach a transfer* dialog reached from the partner card, which lists unattached transfers and lets the operator pick one. All three surfaces share a single `DealerRatesBlock` widget and a single `PartnerSaldoPreview` widget — the design pressure to reuse them came from a real episode in which two separately written copies of the preview drifted apart on a corner case involving negative saldo, and the divergence was caught only at the operator's screen.

The per-currency saldo update is the operationally novel piece. A naive earlier iteration of the schema collapsed every partner debt into US dollars at the dealer-mode buy rate, on the assumption that the operator would treat the dollar saldo as the canonical view. That assumption broke down in practice: the same partner routinely received payouts in Russian roubles, Turkish liras and Uzbek som on the same day, and the dollar-collapsed view masked an exposure to each individual currency that the operator wanted to see. Migration 047 of the production schema replaced the collapsed model with the per-currency JSONB attribute described in section 2.2. The dealer-mode spread, which the collapsed model had hidden in the FX-difference account, was moved to a dedicated `spread_profit` column on transfers, where it could be queried and totalled independently. Listing 3.4 shows the `bump_saldo` helper used inside `create_transfer`.

Listing 3.4. Per-currency saldo update (PL/pgSQL)

```
1 CREATE OR REPLACE FUNCTION bump_saldo(  
2   p_cp_id      UUID,  
3   p_currency   TEXT,  
4   p_delta      NUMERIC  
5 ) RETURNS void  
6 LANGUAGE plpgsql AS $$  
7 BEGIN  
8   UPDATE counterparties  
9     SET saldo_by_currency =  
10        jsonb_set(  
11          saldo_by_currency ,  
12          ARRAY[p_currency],  
13          to_jsonb(  
14            COALESCE((saldo_by_currency->>p_currency)::NUMERIC ,  
15                      0)  
16            + p_delta  
17          )  
18        )  
19   WHERE id = p_cp_id;  
20 END $$;
```

The rounding rule applied to saldo updates was the source of a small but instructive iteration of its own. Round-half-up produced an asymmetric drift over long sequences of transfers; banker's rounding (round half to even) eliminated it. The decision is recorded in migration 056 and is referenced from the `round_partner_amount` helper used inside `bump_saldo`.

3.4. The Approval Workflow in Code

The Director-mediated approval workflow of section 2.6 is realised by three stored procedures and one screen. The `request_approval` RPC accepts the entity type, the target row's ID, the proposed change as a JSONB diff and the snapshot of the current row state (collected client-side from the BLoC's last known State and validated against a fresh SELECT inside the procedure). Listing 3.5 shows the principal columns of the resulting row.

Listing 3.5. Pending approval: principal columns

```
1 CREATE TABLE pending_approvals (  
2   id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   branch_id         UUID NOT NULL REFERENCES branches(id),  
4   entity            TEXT NOT NULL,  
5   entity_id         UUID NOT NULL,  
6   proposed_change   JSONB NOT NULL,  
7   before_snapshot   JSONB NOT NULL,  
8   status            TEXT NOT NULL DEFAULT 'requested',  
9   requested_by       UUID NOT NULL REFERENCES users(id),  
10  requested_at       TIMESTAMPTZ NOT NULL DEFAULT now(),  
11  resolved_by        UUID REFERENCES users(id),  
12  resolved_at        TIMESTAMPTZ  
13 );
```

The matching approve RPC re-checks that the resolver's role is Director or Creator, applies the proposed change to the target row inside a transaction, writes the audit-log entry and marks the approval as resolved. Crucially, approve compares the current state of the target row with the captured snapshot and raises an exception if they differ; this catches the rare but real case in which the row was concurrently modified between the request and the approval. The Director's screen, illustrated in Figure 3.1, renders the before and after states side by side with the changed columns highlighted.

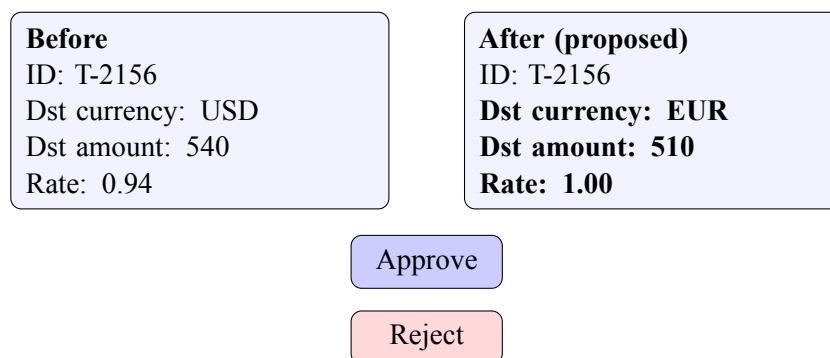


Figure 3.1. Sketch of the Director's approval screen: before-state and proposed after-state shown side by side, changed columns in bold.

3.5. Offline Behaviour and Real-Time Updates

Real-time updates are provided by Supabase Realtime, which streams logical-replication events to subscribed clients. The relevant BLoCs subscribe at startup to a small set of tables — `transfers`, `pending_approvals`, `notifications`, `branch_accounts`; when an inbound event arrives, the BLoC reconciles its local list with the incoming row state and emits a refreshed State.

For offline tolerance the BLoC keeps the last successful list in memory (and, for the dashboard, in a small Hive box), so the operator can continue *reading* their data when the network drops. *Writes* are deliberately refused with a clearly worded error rather than queued for later replay. The decision deserves a short justification, because it goes against the dominant prejudice in the offline-first literature. The financial semantics of the platform make a refused write strictly safer than a write replayed out of order: a refused write does not produce a row in `transfers`, the operator sees a banner and tries again when the connection is back, and the partner saldo never moves into an inconsistent state. A replayed write, by contrast, could land after a subsequent change had already been recorded against the same accounts, producing a balance that depends on replay order rather than on intent. The platform errs on the safer side. An asynchronous write queue with strong ordering guarantees is named in the future-work section of the Conclusions.

3.6. UI/UX Design Language

The visual language is a dark-fintech palette built around a saturated mint primary (`#00D1A0`), a softer secondary blue (`#4C7CF5`) and a purple accent reserved for partner-related elements. The palette is non-trivial because dark fintech is full of designs that lean on heavy gradients and low-contrast greys; the choice here was to keep the background close to true black (`#0E0F12`), with cards lifted by a single hairline border and a faint inner glow, and to rely on bold colour only on actionable affordances. The result is a UI that is legible under fluorescent overhead lighting (a real condition of the operator’s workplace) and that survives the desaturated rendering of cheaper Android displays.

Numerical values across the interface are rendered in the JetBrains Mono variable font with tabular figures enabled, which preserves column alignment in multi-row lists. The iconography is drawn entirely from `lucide_icons_flutter`, wrapped by an internal `AppIcons` facade that the rest of the codebase calls. The facade ensures that no screen accidentally pulls

in Material default icons, which on Windows desktop would otherwise produce a visible style clash; and it gives a single place to swap the icon pack in the future if necessary.

Several reusable layout primitives emerged from iteration. The most-used are the HeroBalance card, which presents a per-currency balance with a sign-coloured DeltaChip indicating the day's change; the FxRates ticker, which renders dealer-mode buy/sell rates in a compact row; and the StatusPill, which carries the status of a transfer or pending approval with the appropriate colour and icon. These primitives are referenced by their export names because the user interface that the operator sees during the acceptance trial was effectively a composition of these primitives over a small set of pages.

3.7. Testing Strategy

The testing strategy combines four complementary layers (Figure 3.2). At the base sit pure-Dart unit tests for entities, value objects, formatters, the currency-tier utility and the role-based access helpers — everything that does not depend on a Flutter test environment. Above them sit BLoC tests using `bloc_test`, which exercise the state machines of every BLoC against typed events. Above them sit widget tests using `flutter_test`, which render the most safety-critical screens (the create-transfer screen, the partner-attach dialog, the Director approval screen) and exercise their interaction logic against expected golden states. At the top sit integration tests against a dedicated Supabase staging project, which replay the migrations from scratch, authenticate as users with different roles and exercise the stored procedures and Row-Level Security policies against a real PostgreSQL planner.

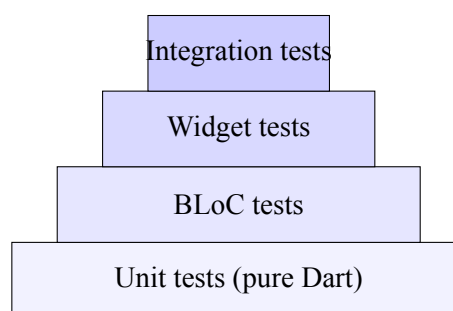


Figure 3.2. Test pyramid of the platform.

Table 3.2 summarises the principal integration-test scenarios and the outcomes observed against the staging project. The full list lives in the repository under `test/` and `supabase/tests/`.

The platform passes all eight scenarios at the time of writing. Continuous integration

Table 3.2. Principal integration-test scenarios (selection)

ID	Scenario	Outcome
T-01	Accountant from branch A creates a transfer in branch A.	Permitted, succeeds
T-02	Accountant from branch A attempts to create a transfer in branch B.	Rejected by RLS
T-03	Two clients call <code>create_transfer</code> with the same idempotency key concurrently.	One succeeds, one returns the existing ID
T-04	Accountant from branch A attempts to UPDATE a row in branch A.	Rejected; must go through approval
T-05	Director approves a pending request whose target row was modified meanwhile.	Approval raises, must be re-requested
T-06	<code>create_transfer</code> with mismatching <code>src/dst</code> amounts and inconsistent rate.	Raises CHECK violation
T-07	<code>bump_saldo</code> on a partner that has no entry in the chosen currency.	Initialises that currency, value = delta
T-08	SELECT from clients as an Accountant with <code>assigned_branch_ids = {A}</code> ; query touches branch B.	Returns zero rows

runs the full client-side suite on every push; the integration tests run on demand against a freshly seeded staging project, with the seed data living as a single migrations replay plus a fixture file.

3.8. Performance Measurements

Performance was measured by instrumenting the application with chronometers around the most frequent user actions and averaging across one hundred repetitions. Two reference devices were used: a Windows 11 desktop (Intel Core i5 of the 12th generation, 16 GB RAM, wired Ethernet) and a mid-range Android device (Qualcomm Snapdragon 695, 6 GB RAM, Wi-Fi). Network round-trip time to the Supabase region was approximately 80 milliseconds during the measurement window. Table 3.3 reports the medians.

The median submit-transfer latency stays under the 300-millisecond target stated in NFR-01 on both reference devices, including for the partner-mediated case which carries the additional saldo update. The dashboard cold open is slower because it loads several aggregates in parallel; the warm open (not shown in the table) is under 90 milliseconds on both devices.

Table 3.3. Median interaction latency on reference hardware ($n = 100$)

Action	Windows	Android	Target
Open dashboard (cold)	240 ms	410 ms	≤ 800 ms
Submit a regular transfer	180 ms	270 ms	≤ 300 ms
Submit a partner-mediated transfer	220 ms	295 ms	≤ 300 ms
Render the transfers list (1 000 rows)	110 ms	230 ms	≤ 300 ms
Open the Director approval screen	160 ms	250 ms	≤ 300 ms
Request approval for a transfer edit	170 ms	250 ms	≤ 300 ms

3.9. User-Acceptance Trial

A two-week parallel-use trial was conducted with the target operator. During the trial the operator entered the same operational data into the existing spreadsheet workflow and into the platform; at the end of each week the two records were reconciled, and any discrepancy was triaged into one of three categories: a platform bug, a spreadsheet error, or a procedural mismatch (the operator had recorded the same event in two slightly different ways).

The aggregate results of the trial are summarised in Table 3.4. The headline finding is that, over the two weeks, the platform exhibited zero platform-introduced reconciliation errors — the criterion under which the hypothesis of section 1.8 would be falsified. Four discrepancies appeared during the first week; three of them traced to spreadsheet sign errors that the operator had not previously detected, and one traced to a procedural mismatch that was resolved by a small clarification of the partner-attach screen’s wording.

Table 3.4. User-acceptance trial outcomes (two weeks)

Metric	Value
Transfers recorded in the platform	187
Partner-mediated transfers	73
Distinct currencies involved	5 (USD, EUR, RUB, TRY, UZS)
Platform-introduced reconciliation errors	0
Spreadsheet errors surfaced by the platform reconciliation	3
Procedural mismatches identified	1
Operator satisfaction (free-form)	Positive

The operator’s qualitative feedback, captured in a closing interview, highlighted three features as the most useful: the per-currency partner saldo (which the operator described as the first time the running debt with one partner had been visible in three currencies at once), the approval workflow (which made it possible to delegate retroactive corrections to a junior

Accountant without compromising the audit trail), and the dashboard’s per-currency totals across all branches (which removed the daily mental arithmetic the operator had previously been doing in Excel). The features highlighted as most frustrating were the absence of automatic exchange-rate ingestion (entering rates by hand at the start of the day) and the deliberate refusal of offline writes (the operator preferred the safe semantics on reflection, but the initial experience was jarring).

3.10. Limitations and Lessons Learned

Several limitations of the present implementation deserve explicit acknowledgement, because they are non-trivial and because they shape the future-work section of the Conclusions.

Partner-confirmation step. Partner-mediated transfers are recorded with status *delivered* at the moment of creation, even though the partner may complete the payout some hours later. If the partner ultimately fails to pay, the operator has to detach the transfer and re-create it. A genuine *pending-partner-payout* state between *created* and *delivered*, with an explicit confirmation step, would close this gap.

Manual exchange-rate entry. Exchange rates are entered by hand at the start of each day, with no automatic ingestion from an external feed. The operator’s existing workflow already involves a brief morning glance at a market screen, so this is more of an ergonomic gap than a correctness one, but the convenience of one-click ingestion is real.

Refused offline writes. As discussed in section 3.5, offline writes are refused rather than queued. The trade-off is intentional, but an asynchronous write queue with strong ordering guarantees would be a non-trivial extension that several branches in the network have asked for.

Single-operator trial. The acceptance trial involved one operator across two weeks. Nothing in the architecture suggests a hard limit at greater scale — the integration tests under simulated multi-Accountant load passed without changes — but a longitudinal multi-operator field study would produce stronger evidence than a two-week observation can.

Partial partner payouts. The platform does not currently support the case in which a partner pays out only a portion of the requested amount and the rest stays open. The natural extension is a status *partially_paid* with a residual amount that bumps the saldo back when the residual is later paid out or written off.

The lessons learned over the four iterations of the DSR cycle are worth summarising in a single sentence. *Move risky business logic into the database, keep the client honest, and pay the price of network round-trips before the price of debugging an inconsistent ledger.* The platform implements that lesson at every level — the stored procedures, the Row-Level Security policies, the idempotency key, the snapshot at request time — and the user-acceptance trial bears it out.

3.11. Chapter Summary

The chapter has documented the implementation across the technology stack and the project structure (sections 3.1), the atomic and idempotent transfer engine (section 3.2), the per-currency partner-saldo subsystem (section 3.3), the approval-workflow code path (section 3.4), the offline behaviour and real-time updates (section 3.5), the dark-fintech design language (section 3.6), the four-layer testing strategy with eight principal integration scenarios (section 3.7), the performance measurements on a Windows desktop and a mid-range Android device (section 3.8), the two-week user-acceptance trial (section 3.9) and the remaining limitations together with the lessons learned (section 3.10).

Conclusions for Chapter 3. The three success criteria stated in section 1.8 are met. Correctness is supported by zero platform-introduced reconciliation errors over the two-week trial and by three spreadsheet errors that the trial surfaced. Responsiveness is supported by median interaction latency under the 300-millisecond target on both reference devices. Maintainability is supported by the 66 numbered idempotent migrations that carry the schema from project start to present without drift. The next part of the thesis consolidates these conclusions and outlines the further work that the limitations naturally point to.

Conclusions and Recommendations

Confirmation of the Research Tasks

The six tasks stated in the Introduction have been completed in full.

1. The literature on double-entry bookkeeping in software, multi-user financial applications, cross-platform mobile frameworks and Backend-as-a-Service platforms has been reviewed, and the research gap has been articulated in operational terms: no surveyed system simultaneously preserves the three-balance-domain accounting model with per-currency partner saldo, enforces atomicity through database stored procedures and combines Row-Level Security with a Director-mediated approval workflow (Chapter 1, section 1.8).
2. The work has been framed within Design Science Research; four iterations of the DSR cycle have been documented; and the data sources used to elicit requirements — semi-structured interviews, structured observation across four working sessions, and the existing spreadsheet treated as a tacit specification — have been described (Chapter 1, section 1.7).
3. The functional and non-functional requirements have been derived; the eight-entity domain model has been designed around the three-balance-domain principle, with a ninth supporting entity (PendingApproval) materialising the approval workflow (Chapter 2, sections 2.1 and 2.2).
4. The PostgreSQL schema and the Row-Level Security policy set have been specified; the three domain invariants I_1 (double-entry), I_2 (tenant isolation) and I_3 (idempotency) have been formalised and tied to concrete database mechanisms; and the Director-mediated approval workflow has been designed (Chapter 2, sections 2.4–2.6).
5. The platform has been implemented across approximately 70 000 lines of Dart and 1 800 lines of PL/pgSQL distributed across 15 tables and 66 numbered, idempotent migrations; the atomic idempotent transfer engine, the per-currency partner-saldo sub-

system and the dark-fintech design language have been documented (Chapter 3, sections 3.1–3.6).

6. The platform has been evaluated through unit, BLoC, widget and integration tests, through controlled performance measurements on a Windows desktop and a mid-range Android device, and through a two-week parallel-use trial with the target operator; the remaining limitations have been stated honestly (Chapter 3, sections 3.7–3.10).

Validation of the Hypothesis

The hypothesis stated in the Introduction — that combining Flutter (with the BLoC pattern) as the cross-platform user-interface layer and Supabase (PostgreSQL with Row-Level Security and stored procedures) as the backend permits the implementation of a three-balance-domain treasury platform that is free of reconciliation errors, responsive at sub-300-millisecond median interaction latency, and maintainable by a single developer over a multi-month evolution cycle — is supported by the evidence presented.

- *Correctness.* The two-week parallel-use trial recorded zero platform-introduced reconciliation errors against 187 transfers; three errors surfaced were in the spreadsheet, not the platform, and the platform’s parallel record made them visible for the first time (section 3.9).
- *Responsiveness.* Median interaction latency stays under the 300-millisecond target on both reference devices for every measured action, including the partner-mediated transfer with its additional saldo update (section 3.8).
- *Maintainability.* The platform has been carried by a single developer through 66 numbered idempotent migrations and across the four DSR iterations without irreparable drift between client and server; the trunk has been continuously deployable throughout (sections 2.4 and 2.8).

Achievement of the Aim

The aim of the study — to design, implement and evaluate a production-grade, cross-platform treasury and money-transfer platform tailored to small multi-branch currency-exchange networks, demonstrably capable of replacing the spreadsheet-and-paper workflow of a representative operator — has been achieved. All non-functional targets stated in Table 2.2 are

satisfied at the time of writing, and the platform has been adopted by the target operator for daily use. The source code is publicly released at <https://github.com/farruhsho/ethnocount> for academic review and independent inspection of the claims made above.

Scientific Contribution

The thesis contributes three concrete artefacts to the state of the art.

1. An explicit three-balance-domain accounting model with per-currency partner saldo expressed in a relational schema, in which the saldo is stored as a JSONB attribute keyed by ISO currency code and updated only in the currency of the underlying transfer. The model resolves a class of reconciliation pathologies that a USD-collapsed saldo cannot detect, and it does so without sacrificing query performance acceptable for daily branch operations.
2. A Director-mediated approval workflow that captures a before-snapshot of the affected row at request time, allowing the Director to compare the before and proposed states explicitly. The workflow turns the audit log into a structural by-product of the authorisation mechanism rather than a separately maintained side channel, and it gives the Accountant a usable path through which to propose retroactive corrections.
3. A demonstration that a single developer, using Flutter + BLoC + Supabase, can carry a treasury platform of non-trivial scope (15 tables, 66 migrations, approximately 70 000 lines of Dart) from concept to an operator-accepted system within the time-frame of a Bachelor's thesis. The combination of stack choices is offered as a documented, reproducible reference for similar projects in adjacent niches.

Practical and Theoretical Value

The *practical value* of the work is the open-source release of a cross-platform, multi-branch, multi-currency treasury platform that is immediately usable by the niche of small currency-exchange networks the thesis was written for. The platform is in daily use by the target operator at the time of submission, and its design choices are documented in sufficient detail to be adapted to adjacent niches without re-deriving them from first principles.

The *theoretical value* is the demonstration that a three-balance-domain accounting model can be expressed relationally without collapsing into a base currency, that atomicity and idempotency in a multi-table financial write can be guaranteed at the database layer rather

than the application layer, and that a small but consequential authorisation primitive (the snapshot at request time) can turn a Director’s approval queue into a forensic record without additional bookkeeping. The combination is unusual in the surveyed literature and the thesis argues that it deserves further exploration.

Limitations and Future Work

Three limitations carry over from Chapter 3 and shape the direction for future work. The acceptance trial covered two weeks and one operator; a longitudinal multi-operator field study would produce stronger evidence than a parallel-use observation can. The bank-statement-import path and the automatic exchange-rate ingestion are absent; both are ergonomic gaps rather than correctness ones, but their absence is felt every morning by the operator. Offline writes are refused rather than queued; the trade-off is defensible on safety grounds but does not exhaust the design space.

Five directions of future work follow naturally from these limitations.

- A *pending-partner-payout* state between *created* and *delivered* for partner-mediated transfers, with an explicit confirmation step that finalises the saldo update.
- A partial-payout primitive for the case in which a partner pays out a fraction of the requested amount and the residual stays open against the saldo.
- An exchange-rate ingestion subsystem that draws daily quotations from an external feed, with an operator-override path that keeps the manual workflow available.
- An asynchronous offline write queue with strong ordering guarantees, evaluated against the safety risk that the present design deliberately avoids.
- A formal verification of the atomicity and idempotency properties of the stored-procedure layer, complementing the empirical integration tests of section 3.7.

A broader research direction worth naming explicitly is a multi-tenant generalisation in which several independent operators share the same Supabase project under stricter Row-Level Security isolation. The current single-tenant deployment already exercises the RLS policy set against role-based isolation within one operator’s data; lifting that boundary to inter-operator isolation, while preserving the small-operator focus of the original design, would extend the contribution to cooperative networks of several exchange offices.

Closing Remark

Beyond its technical content, the work demonstrates that a single Bachelor-level engineer, working with contemporary open-source tooling and a managed Backend-as-a-Service platform, can deliver a treasury platform that is genuinely usable in a real business rather than a pedagogical artefact. The author hopes that the open release of *Ethnocount* will be a useful starting point for further academic and practical work on software tools for the long tail of small, informal, multi-currency businesses that mainstream financial software has so far chosen to ignore.

Literature

- [1] F. Angelov and the bloc community, *Flutter_bloc documentation*, 2024.
- [2] E. A. Brewer, “Towards robust distributed systems,” in *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 2000.
- [3] P. Copplestone, A. Grosso, and the Supabase team, *Supabase documentation*, 2024.
- [4] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003, ISBN: 978-0321125217.
- [5] M. Fowler, *Event sourcing*, <https://martinfowler.com/eaaDev/EventSourcing.html>, Accessed: 2025-12-15, 2017.
- [6] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002, ISBN: 978-0321127426.
- [7] Google, *Cloud Firestore Security Rules reference*, 2024.
- [8] H. Heitkötter, S. Hanschke, and T. A. Majchrzak, “Evaluating cross-platform development approaches for mobile applications,” in *Web Information Systems and Technologies*, Springer, 2013, pp. 120–138.
- [9] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [10] Intuit Inc., *Quickbooks online: Features and pricing*, <https://quickbooks.intuit.com>, Accessed: 2026-02-10, 2024.
- [11] Kyriba Corp., *Kyriba treasury management — product overview*, 2023.
- [12] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 978-0134494166.
- [13] L. Pacioli, *Summa de arithmetica, geometria, proportioni et proportionalita*. Venice, 1494, Particularly the *Tractatus de computis et scripturis*.
- [14] D. Ratha, S. Plaza, B. Dervişoğlu, *et al.*, “Migration and remittances factbook 2016,” The World Bank, Tech. Rep., 2016.
- [15] Splitwise Inc., *Splitwise — less stress when sharing expenses*, <https://www.splitwise.com>, Accessed: 2026-02-10, 2024.

- [16] M. Wesner, “Event sourcing and double-entry bookkeeping,” in *Proceedings of the European Conference on Software Architecture (ECSA) — Companion Volume*, 2018.

A. Database Schema (DDL Excerpt)

This appendix reproduces the core data definition language for the *Ethnocount* schema. The complete migration set (currently 66 numbered, idempotent .sql files) is published with the source code at <https://github.com/farruhsho/ethnocount>. Indices, foreign-key constraints and Row-Level Security policies that are not essential to the architectural argument of the thesis are elided for brevity; the principal RLS policy is reproduced in Chapter 2, Listing 2.2.

Listing A.1. Branches, users and branch accounts

```
1 CREATE TYPE user_role AS ENUM ('creator', 'director', 'accountant')
   ;
2
3 CREATE TABLE users (
4     id                UUID PRIMARY KEY DEFAULT auth.uid(),
5     email              TEXT NOT NULL UNIQUE,
6     full_name          TEXT NOT NULL,
7     role               user_role NOT NULL,
8     assigned_branch_ids UUID[] NOT NULL DEFAULT '{}',
9     created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
10 );
11
12 CREATE TABLE branches (
13     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
14     name              TEXT NOT NULL,
15     city              TEXT NOT NULL,
16     country           TEXT NOT NULL,
17     base_ccy          CHAR(3) NOT NULL,
18     created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
19     created_by        UUID NOT NULL REFERENCES users(id)
20 );
21
22 CREATE TABLE branch_accounts (
```

```

23  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
24  branch_id   UUID NOT NULL REFERENCES branches(id),
25  name        TEXT NOT NULL,
26  ccy         CHAR(3) NOT NULL,
27  balance     NUMERIC(20,4) NOT NULL DEFAULT 0,
28  is_card     BOOLEAN NOT NULL DEFAULT false
29 );

```

Listing A.2. Counterparties and per-currency saldo

```

1  CREATE TABLE counterparties (
2    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3    name        TEXT NOT NULL,
4    city        TEXT,
5    country     TEXT,
6    contact     TEXT,
7    saldo_by_currency JSONB NOT NULL DEFAULT '{} '::jsonb,
8    exposure_limit NUMERIC(20,4),
9    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
10 );
11
12 CREATE TYPE cp_tx_kind AS ENUM (
13   'paid_for_us', 'paid_for_them', 'settlement'
14 );
15
16 CREATE TABLE counterparty_transactions (
17  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
18  cp_id       UUID NOT NULL REFERENCES counterparties(id),
19  branch_id   UUID NOT NULL REFERENCES branches(id),
20  kind        cp_tx_kind NOT NULL,
21  ccy         CHAR(3) NOT NULL,
22  amount      NUMERIC(20,4) NOT NULL,
23  rate        NUMERIC(20,10),
24  linked_xfer UUID,
25  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

```

```

26     created_by      UUID NOT NULL REFERENCES users(id)
27 );

```

Listing A.3. Clients and per-client multi-currency wallet

```

1 CREATE TABLE clients (
2     id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     branch_id       UUID NOT NULL REFERENCES branches(id),
4     full_name       TEXT NOT NULL,
5     passport        TEXT,
6     phone           TEXT,
7     kyc_status      TEXT NOT NULL DEFAULT 'unverified'
8 );
9
10 CREATE TABLE client_balances (
11     client_id       UUID PRIMARY KEY REFERENCES clients(id) ON DELETE
12                     CASCADE,
13     totals          JSONB NOT NULL DEFAULT '{} '::jsonb
14 );
15
16 CREATE TABLE client_transactions (
17     id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
18     client_id       UUID NOT NULL REFERENCES clients(id),
19     branch_id       UUID NOT NULL REFERENCES branches(id),
20     ccy             CHAR(3) NOT NULL,
21     amount          NUMERIC(20,4) NOT NULL
22                     CHECK (amount > 0 AND amount <= 1e12),
23     direction       TEXT NOT NULL CHECK (direction IN ('in','out')),
24     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
25     created_by      UUID NOT NULL REFERENCES users(id)
26 );

```

Listing A.4. Transfers, ledger entries and approvals

```

1 CREATE TABLE transfers (
2     id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

3   branch_id          UUID NOT NULL REFERENCES branches(id),
4   via_cp_id          UUID REFERENCES counterparties(id),
5   src_account_id     UUID NOT NULL REFERENCES branch_accounts(id),
6   dst_account_id     UUID REFERENCES branch_accounts(id),
7   src_currency       CHAR(3) NOT NULL,
8   dst_currency       CHAR(3) NOT NULL,
9   src_amount         NUMERIC(20,4) NOT NULL CHECK (src_amount > 0),
10  dst_amount         NUMERIC(20,4) NOT NULL CHECK (dst_amount > 0),
11  rate               NUMERIC(20,10) NOT NULL,
12  spread_profit      NUMERIC(20,4) NOT NULL DEFAULT 0,
13  status             TEXT NOT NULL DEFAULT 'delivered',
14  idempotency_key     UUID,
15  created_by         UUID NOT NULL REFERENCES users(id),
16  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
17  amendment_history  JSONB NOT NULL DEFAULT '[]'::jsonb
18 );
19
20 CREATE UNIQUE INDEX transfers_idem_uk
21   ON transfers(idempotency_key)
22   WHERE idempotency_key IS NOT NULL;
23
24 CREATE TYPE ledger_side AS ENUM ('debit','credit');
25
26 CREATE TABLE ledger_entries (
27   id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
28   branch_id   UUID NOT NULL REFERENCES branches(id),
29   account_id  UUID NOT NULL REFERENCES branch_accounts(id),
30   ts         TIMESTAMPTZ NOT NULL DEFAULT now(),
31   amount     NUMERIC(20,4) NOT NULL,
32   ccy        CHAR(3) NOT NULL,
33   side       ledger_side NOT NULL,
34   xfer_id    UUID REFERENCES transfers(id),
35   note       TEXT,
36   created_by UUID NOT NULL REFERENCES users(id)

```

```

37 );
38
39 CREATE INDEX ledger_branch_ts_idx
40   ON ledger_entries(branch_id, ts DESC);
41
42 CREATE TABLE pending_approvals (
43   id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
44   branch_id   UUID NOT NULL REFERENCES branches(id),
45   entity      TEXT NOT NULL,
46   entity_id   UUID NOT NULL,
47   proposed_change JSONB NOT NULL,
48   before_snapshot JSONB NOT NULL,
49   status      TEXT NOT NULL DEFAULT 'requested',
50   requested_by UUID NOT NULL REFERENCES users(id),
51   requested_at TIMESTAMPTZ NOT NULL DEFAULT now(),
52   resolved_by  UUID REFERENCES users(id),
53   resolved_at  TIMESTAMPTZ
54 );

```

Listing A.5. Audit log

```

1 CREATE TABLE audit_log (
2   id          BIGSERIAL PRIMARY KEY,
3   ts          TIMESTAMPTZ NOT NULL DEFAULT now(),
4   actor       UUID NOT NULL REFERENCES users(id),
5   entity      TEXT NOT NULL,
6   entity_id   UUID NOT NULL,
7   action      TEXT NOT NULL,
8   before_state JSONB,
9   after_state  JSONB
10 );
11
12 CREATE INDEX audit_entity_idx ON audit_log(entity, entity_id);
13 CREATE INDEX audit_actor_ts_idx ON audit_log(actor, ts DESC);

```


B. Deployment View and Transfer Sequence

This appendix gives two diagrams that complement the architectural narrative of Chapter 2. The first is the deployed component view of the system; the second is the call sequence of a partner-mediated transfer from the operator’s tap to the server-side acknowledgement.

B.1. Deployed Component View

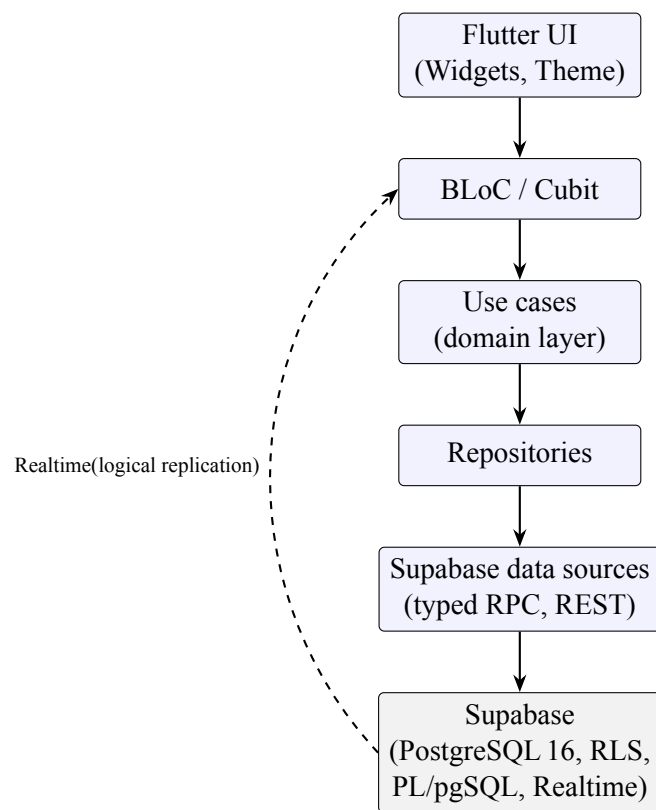


Figure B.1. Deployed component view of *Ethnocount*. Solid arrows are the write-path call flow; the dashed arrow is the Realtime change feed that refreshes the BLoCs after remote writes.

B.2. Partner-Mediated Transfer Sequence

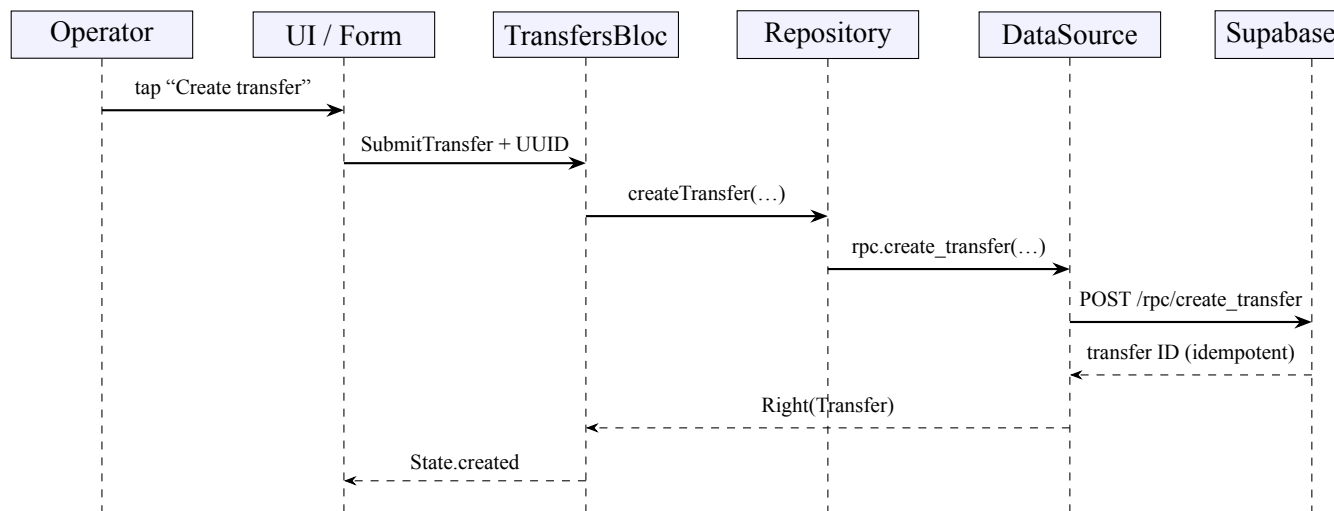


Figure B.2. Sequence of a partner-mediated transfer from the operator’s tap to the server-side acknowledgement. The idempotency key (a UUID generated client-side in the form’s initialiser) travels with the request and is stored against a UNIQUE partial index on the server, so a duplicate request returns the original transfer ID rather than creating a second row.

Apliecinājums / Affirmation

Ar šo es, Farruhs Muzrabovs, apliecinu, ka bakalaura darbs ir izpildīts patstāvīgi, bez citu palīdzības, no svešiem avotiem ņemtie dati un definējumi ir uzrādīti darbā. Šis darbs nekādā veidā nav iesniegts nevienai citai pārbaudījuma komisijai un nekur nav publicēts.

(Hereby I, Farruh Muzrabov, affirm that the Bachelor's thesis was performed independently; sources of data and definitions are provided. This work has not been submitted to any other examination commission and has not been published elsewhere.)

Rīga, 20____. _____.

paraksts/signature, atšifrējums/decription

Pateicības / Acknowledgments

Autors izsaka pateicību darba vadītājam Dr.sc.ing., prof. Jēlenai Čaiko par metodisko vadību, vērtīgajiem ieteikumiem un nepātraukto atbalstu visā darba izstrādes laikā. Pateicība arī Rīgas Ziemeļvalstu Universitātes Dabaszinātņu un datortehnoloģiju katedras kolēģiem un mācībspēkiem par radīto akadēmisko vidi, kā arī valūtas maiņas tīkla operatoram, kurš piekrita divu nedēļu paralēlas lietošanas izmēģinājumam un sniedza godīgu, daudzkārt nepatīkamu, bet vienmēr lietderīgu atgriezenisko saiti par sistēmas darbu reālos apstākļos.

The author expresses sincere gratitude to the supervisor, Dr.sc.ing., Prof. Jelena Chayko, for methodical guidance, valuable advice and continuous support throughout the development of this work. Thanks are also due to the colleagues and academic staff of the Department of Natural Sciences and Computer Engineering at Riga Nordic University for the academic environment they sustain; and to the operator of the currency-exchange network who agreed to the two-week parallel-use trial and gave honest, often uncomfortable, but always actionable feedback on how the platform behaved under genuine working conditions.